

# claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_b3a30c42-346\agent-a393c0ca3522feefd.jsonl`  
- SHA-256: `1c03948f6554bcb748f720c5b0f42c9f9539052d81f73c135be5b2adba75b5c7`  
- Source modified: `2026-06-16T02:27:36+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_b3a30c42-346`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

## Transcript

### 1. user (2026-06-16T02:24:52.619Z)

Map the pipeline's data contracts and the natural VIDEO-FREE seam(s). Read: docs/CODE\_MAP.md (especially data-contract sections), docs/PER\_VIDEO\_OUTPUT\_LAYOUT.md, docs/HOW\_RALLY\_DETECTION\_WORKS.md, docs/MULTI\_SIGNAL\_FUSION\_PLAN.md, backend/pipeline/segmenters/base.py, backend/pipeline/detectors/base.py, backend/pipeline/detectors/wasb\_runner.py; then grep+read the fusion / rally-gate code (search 'rally\_gate'), the stitcher (search 'stitch'), the candidate-segments persistence (search 'candidate' in backend/infrastructure/database.py and backend/pipeline), and backend/utlis/hashing.py. ANSWER with concrete field names: (1) The end-to-end stage sequence from video to final highlight segments, naming each intermediate artifact and its exact shape/format (trajectory CSV columns; candidate-segment record fields; per-signal feature columns for person/audio/court; final segment record). (2) Precisely WHERE video/frames are required vs where everything downstream is pure deterministic transformation. (3) The best seam(s) to FREEZE for video-free regression and exactly what must be captured at each. (4) Sources of non-determinism downstream of that seam (randomness, timestamps, float, ordering, env/library).

### 2. assistant (2026-06-16T02:24:55.688Z)

I'll map the pipeline's data contracts and find the video-free seams. Let me start by reading the key docs and code files in parallel.

### 3. user (2026-06-16T02:24:57.053Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-151 of 379 total (37029 tokens, cap 25000). Call Read with offset=152 limit=151 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```
1      # CODE_MAP.md  
2  
3      **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-  
hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a  
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.  
4  
5      **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this  
doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure  
(this revision follows #40).  
6  
7      ? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust  
git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no  
FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via  
`segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not  
basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec`  
defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**;
```

```

`match_intervals` requires real overlap (iou>0); `run_regression` baseline lives in tracked
`eval_baselines/` and **exits 3 on a missing baseline**; the API validates
`video_path`/`output_filename`. New tests: `test_api_endpoints`, `test_compiler`,
`test_wasb_runner`, `test_reingest_safety`, `test_detector_keying`, `test_config_wasb`,
`test_path_safety`, `test_reliability_hardening` (+ collector `test_licensing_gate`,
`test_import_guard`, `test_cvat_robustness`).
8
9     **LEGEND**
10    - `@path` = file (always repo-relative)
11    - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
12    - `->` = dataflow / call / "produces"
13    - `$` = data contract (canonical shape)
14    - `?` = gotcha / footgun
15    - `?` = registry / plugin extension point
16    - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
17
18    ---
19
20    ## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)
21
22    Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an
existing
23    **registry / extension point** (`?`) over a new top-level file.
24
25    | You're adding? | Put it in? | Notes |
26    |---|---|---|
27    | Backend / runtime code | `backend/<subsystem>/` |
`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`,
`infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most
additions are **register a class in the relevant `?` registry**, not a new module tree. |
28    | **Eval LIBRARY** (importable, pure ? metrics, calibration, a scorer, a feature) |
`backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver.
These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not
scripts; do NOT move them out. |
29    | **Standalone run-driver / one-off entry CLI** (has `__main__`, NOT imported by app
code) | **scripts/** | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`.
Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing
backend, add `scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
30    | **Ops / maintenance tool** (not part of the served app ? cache GC, batch admin) |
`tools/` | Run as `python -m tools.<name>` (it's an importable package). e.g.
`tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested. |
31    | Training code (model fitting) | `training/` | Behind the CI-enforced import wall
(ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`. |
32    | A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a **70%
coverage floor**. Pure/offline + mocked (no GPU/WSL/network). |
33    | A doc (design, plan, living log) | `docs/` | Index it in `docs/README.md`. Only
**terminal-state** (DONE/REJECTED) work moves to `docs/archives/`. |
34    | A throwaway repro / debug script | `scratch/` | **Gitignored** + excluded from
ruff/pytest. Never imported by app code. |
35    | A model artifact (weights + manifest) | `artifacts/<name>/<ver>/` | ADR-008:
**manifest committed** (provenance), **weights gitignored**. |
36    | Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean
checkout); the no-regression gate reads it. |
37    | Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | **Gitignored.**
Per-video isolation is planned in `PER_VIDEO_OUTPUT_LAYOUT.md`. |
38
39    **Canonical files that stay at the repo ROOT** (referenced by CI / packaging / docs ? do
not move):
40    `pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) .
`run_all_tests.py` (CI: `ci.yml`) .
41    `run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) .
`run_eval.py`
42    (grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic
`setup.py` moved to
43    `scripts/doctor.py` (it shadowed the build system); invoke via `python

```

```

scripts/doctor.py` or `indexer --setup`.
44     New standalone run-drivers go in `scripts/`, not the root.
45
46     ---
47
48     ## 1. PIPELINE
49
50     ### 1A. Runtime: video file -> highlight reel
51     ```
52     typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
53                                     @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
54     -> video_id = MD5(whole file)                                     @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
55     -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
56     FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
57     [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
58     -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
59     -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
60     -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
61     -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
62     [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
63     -> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
64     -> select segment ids -> [(start,end)] (+ stitching padding)
65     -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
66     per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
67     ```
68     Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
69     ```
70     P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
71     -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted
via db.add_candidate_segment]
72     -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
73     -> P4 db.add_play_segment + db.link_candidate_to_segment
74     ```
75     Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
76     Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).
77
78     ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
79     ```
80     WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
81     -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
82     -> WasbRunner.run_predict(video, out_dir)
83     stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
84     -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU

```

```

detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
85     -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
86     -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
87     ...
88     (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
89
90     ### 1C. Calibration / eval flow (NO pipeline run unless told)
91     ...
92     GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
93     DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval] stage
? {candidates, final}
94     -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
95     -> harness.evaluate -> EvalReport -> report_to_dict
96         -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
97         -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py
[CI gate]
98     Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
99     Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier) @backend/eval/{calibrate_local,distill_local}.py
100    Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
101    Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py
102    Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
103    ...
104    Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py`
(manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
105
106    ---
107
108    ## 2. SUBSYSTEMS
109
110    ### 2.0 Orchestration & config
111    - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS).
Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig`
(INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces
stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI
summaries may still use `print` for direct output.
112    - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPes.
113    - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_run_job,...)` -> **202 +
job_id**. `::run_job` (worker: mark running -> `_execute_processing` -> done/failed; catches +
logs traceback). `::execute_processing` = the former sync body (hash -> validate -> add_video
-> segment -> `finally:` always `emit_indexer_report`, grafts `response["reports"]`; never
raises from the report); a RAISING segmenter now prune_after-restores pre-run rows before re-
raising. `::get_job` `@GET /api/jobs/{job_id}` -> `{status, progress, video_id, error, result,
reports, ...}` ? job `status` = EXECUTION state (`queued|running|done|failed`); the pipeline
verdict lives in `result["status"]`. `::get_executor` lazy module-global
`ThreadPoolExecutor(max_workers=1)` (single box + Gemini serial-upload learning; tests
monkeypatch it inline). `main()` runs `db.fail_stale_jobs()` at startup (orphaned queued/running
-> failed). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST
/api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy
`import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read). ? CLI `--video-
path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).

```

```

114 - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** chunked upload ? `::upload_init` `@POST
/api/upload/init` (`{filename, total_size_bytes}` -> ext allowlist + size gate ->
`{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY
sequential n, streams to `partial-<id>` under `security.upload_dir`; per-part/total cap breach
-> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble
-> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs
/api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state
(`::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits);
per-user partitioning lands with PR-3. `::download_highlight` `@GET
/api/highlights/{video_id}/{filename}` ? safe-name + `resolve_under_root(output_dir)` then
FileResponse stream (the old /api/compile alert path was browser-unreachable). Config:
`SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95,
allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
115 - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
116 - `main()` sets `global_config.output.output_dir` = args.output_dir` (typed field on
`OutputConfig`); `db_path` = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `runtime_overrides` hack is gone).
117 - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
118 - `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive
`input()` (blocks automation), skips validation. Config via `backend.config.load_config`
(typed); segmenter, padding, and db path all read from the model (no literal fallbacks).
119 - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
`::default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
120 - `@scripts/run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score,
aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as
label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed
`AppConfig` (same object the live pipeline feeds segmenters).
121 - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
**0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run); `::default_db_path` resolves via
`backend.config.load_config`.
122
123 - `@backend/config/_init_.py` ? ? canonical config import surface. Re-exports
`load_config`, `save_default_config` (from `loader`) + `AppConfig`, `Config`, `IndexingConfig`,
`StitchingConfig`, `ValidationConfig` (from `models`); `__all__` = exactly those 7. Use `from
backend.config import load_config, AppConfig`.
124 - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for
defaults**.
125 - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`, `indexing`, `output`, `stitching`
(`default_factory`). `model_config` = {extra:'allow', validate_assignment:True}. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
126 - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real
runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.
127 - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested
`tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `gemini`. ?
`::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ?
a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`).
128 - `::ValidationConfig` (`min_resolution_width=1280`, `min_resolution_height=720`, `min_fps=29.9`,
`min_blur_threshold=20.0`, `max_scene_cuts_per_minute=0.5`, nested `relevance`);

```

```

`::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
`::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`,
`conda_env='TrackNetV4'`, `weights_path='', `queue_length=5`, `velocity_threshold=0.02`).
`::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`,
`output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).
`::StitchingConfig`
(`begin_padding=0.5`, `end_padding=0.5`, `use_cache=False`, `min_shot_count=1`). `::Sport` Literal
of 5 sports. `::Config` = AppConfig alias.
129     - `@backend/config/loader.py` ? load/save with defaults-merge.
130     - `::DEFAULT_CONFIG_PATH` = Path("config.json")` (? CWD-relative, NOT repo-root).
`::get_builtin_defaults` = `AppConfig().model_dump(...)` .
131     - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{*defaults, *file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
132     - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
(used by scripts/doctor.py/--setup). `::get_default_config` ? ? transitional plain-dict alias.
133     - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
134     - `@scripts/doctor.py` (formerly root `setup.py`) ? `::init_default_config` ? delegates
to `backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python>3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA). Run via `python scripts/doctor.py` or `indexer
--setup`.
135
136     ### 2.1 Segmenters (under pipeline/segmenters) ?
137     - `@backend/pipeline/segmenters/base.py`
138     - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`, `description`, `process_video`. $ `process_video(video_id, video_path, player_pool=None,
max_frames=None)` -> List[dict{id,start_time,end_time,duration,metadata}].
139     - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `name` ->
name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-
instantiates + swallows exceptions into "No description available." (a throwing `description` is
masked).
140     - `@backend/pipeline/segmenters/__init__.py` ? imports all 6
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid,fusion_hybrid`) = the de-facto
registration manifest; re-exports them + `segmenter_registry` via `__all__`.
141     - `@backend/pipeline/segmenters/trajectory_hybrid.py::TrajectoryHybridSegmenter` ?
SHARED ENGINE for the trajectory hybrids (2026-06-11 collapse of the copy-paste twins; the "fix
BOTH" footgun is retired): healthcheck gate -> P2 trajectory->windows (+persist) -> P3 provider
handoff -> P4 DB/link; single `::shot_count_from` (old wasb/tracknet shot_count drift gone).
NOT registered itself. ? Subclass contract: `family` (config section
`indexing.<family>.velocity_threshold(0.02)`, `output/<family>` trajectory dir,
`<family>-hybrid-<model>` metadata tag), `display_label` (log lines), `name`/`description`
properties, `build_runner(idx_cfg)` -> (DetectorRunner, detection_params extras). **Two fusion
seams (IDENTITY by default ? twins byte-identical, pinned by the equivalence test):
`::enrich_candidate_stats(cw,points,fps,frame_width,video_path,idx_cfg)`** (base = the 4
motion-key stats dict; computed once into `window_stats`, used for persist + the handoff gate)
**and `::select_for_handoff(windows>window_stats,idx_cfg)`** (base = all indices). ? shared
knobs still YOLO-named (`log_yolo_candidates/store_yolo_candidates`); whole-video single pass;
needs `{PROVIDER}_API_KEY` (missing -> []). Equivalence pinned by
`tests/test_trajectory_hybrid_equivalence.py` (detector identity, shot_count, offsets, + the
base-hook-identity tests ? parametrized over both twins).
142     - `@backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (name
`fusion_hybrid`, **default-OFF** multi-signal fusion P2) ? subclasses the engine;
`family='fusion'`, `display_label='Fusion'`. `::build_runner` selects the substrate via
`indexing.fusion.substrate` ('wasb'|'tracknet') reusing the existing runner+config.
`::enrich_candidate_stats` overrides to ALWAYS add `net_crossings` (Gate A signal, GPU-free via
`rally_gate.gate_windows(..., estimate=self._axis_estimate(points))` ? axis/net cached per
trajectory) and, only when `fusion.cue_flags.person`, the ~5 person features

```

```
(`::_person_features` ? lazy `PersonDetector.detections_per_frame` over the window's ORIGINAL-
video frame range ? `fusion_features.compute_person_features`). `::_select_for_handoff`
overrides to apply served Gate A (`fusion.min_crossings`) ? Gate B (`fusion.min_players`,
person-only) ? default 0/0 ? all windows kept. ? persisted candidates are ALWAYS the full
superset (offline substrate intact); only the handoff/play_segments are gated. ? to A/B vs a
tuned wasb run, set `fusion.velocity_threshold = wasb.velocity_threshold` (the engine reads
`idx_cfg['fusion'].velocity_threshold`). ? person features eager even for gated windows (P3
cleanup). Tests: `tests/test_fusion_hybrid.py`.
143 - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`'wasb_hybrid'`, the LIVE shuttle substrate; MIT, ships pretrained badminton weights) ? thin
subclass: `family='wasb'`, wires `WasbRunner`/`WasbConfig`; `detection_params` extras =
`weights_path` only.
144 - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? thin subclass: `family='tracknet'`, wires
`TrackNetRunner`/`TrackNetConfig`; extras = `weights_path` + `queue_length`.
145 - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name
`'yolo_hybrid'`, no YOLO remains; ADR-005). **Stage-1 detection EXTRACTED to
`detectors/person_detector.py` (fusion P1, no behavior change)** ? the segmenter now
ORCHESTRATES only (chunking, pipelined-vs-sequential P2 prefetch where ffmpeg overlaps
sequential GPU, AI-provider handoff, candidate persist/link) and delegates detection to
`self.person` (`PersonDetector`). Imports `_DEFAULT_DETECTOR` from person_detector for the
`detection_params` snapshot. ? `yolo_tracker`/`bytetrack.yaml` is dead/back-compat; `gemini_*`
config keys are deprecated aliases still honored; still mpy-quarantined for pre-existing
orchestration debt (the torchvision code that caused it left).
146 - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ?
pure-AI, no P2. Chunks are RE-ENCODED + downsampled to `indexing.ai_chunk_scale_width` (default
1280; 0 = legacy stream-copy) before upload ? stream-copied 4K chunks exceed the provider's
`MAX_UPLOAD_MB` and every chunk gets refused ("0 segments / success"); re-encode also fixes
keyframe drift at chunk boundaries. Oversize refusals (provider metrics `oversize_skipped`) are
aggregated -> `logger.error` + `reporting.run_signals` (`oversize_clips_skipped`) for the report
rule. Workers from `indexing.ai_max_workers`. ? chunks only if `duration>chunk_duration`;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
147 - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ?
pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
148
149 ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
150 - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() ->
Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately
matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring
includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on
threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL
de-risk seam (low-regret 6 / review item 1).
151 - `@backend/pipeline/detectors/tracknet_runner.py`
```

#### 4. user (2026-06-16T02:24:57.141Z)

```
1 # Per-video output layout ? one directory per video (design / plan)
2
3 **Status:** **DESIGN ? owner-gated, NOT started.** Proposed convention so processing N
videos
4 yields N self-contained subdirectories instead of one tangled `output/`. Docs-only.
5
6 ## The problem
7 Today every run writes into a **flat `output/`** with filename prefixes ? highlights
8 (`<video>_highlights.mp4`), run logs (`<video>_run.log`), eval CSVs
9 (`<video>_vs_human.csv`),
10 the shared SQLite DB (`sports_indexer.db`), plus **hardcoded shared subdirs** the CLI
11 `--output-dir` does **not** redirect:
12 - `output/wasb/` and `output/tracknet/` ? trajectory CSVs + frame caches
13 (`trajectory_hybrid.py:~176`, `out_dir = os.path.join("output", self.family)` ?
literal `output`).
```

```

13     - `output/chunks_<provider>_handoff/` ? the AI-handoff clip staging (same file).
14
15     So 5 videos produce an interleaved pile, and `--output-dir` only relocates *some*
artifacts
16     (highlights/clips/DB) while trajectories and handoff clips still land in the shared
`output/`.
17     The first isolated run (`output/GX010125_run/`, 2026-06-14) had to be done by hand and
is only
18     *partially* isolated for this reason.
19
20     ## The target
21     ```
22     output/
23     <video_slug>/          # slug = sanitized stem + short video_id (collision-
safe)
24     runlog.txt             # the per-video run log (today: output/<video>_run.log)
25     sports_indexer.db      # per-video DB (or a row-scoped view of a shared DB ?
see below)
26     wasb/<key>_wasb.csv     # the trajectory (today: output/wasb/)
27     handoff/              # AI-handoff clips (today: output/chunks_*_handoff/,
auto-cleaned)
28     rallies.csv           # detected rally [start,end] + shot_count
29     segments/             # per-segment clips / pngs / debug frames
30     highlights.mp4         # the reel
31     report.json / report.md # the observability report
32     ```
33     A directory with 5 videos ? 5 subdirectories, each holding **everything** for that
video.
34
35     ## Design
36     - **`video_slug(video_path, video_id) -> str`** ? a new helper (next to
`utils/hashing.py`):
37     sanitized stem + `video_id[:12]` (the MD5 already used as the cache key), e.g.
38     `GX010125_1080p__89605e80ed01`. Collision-safe (same stem / different bytes ?
different dir).
39     - **Thread one `run_dir` through the pipeline** instead of the literal `"output"`:
40     - `TrajectoryHybridSegmenter.process_video` ? `out_dir = os.path.join(run_dir,
self.family)`
41     and the handoff dir under `run_dir` (replaces the two hardcoded `"output"` joins).
42     - The `wasb_runner`/`tracknet_runner` already take `output_win_dir` ? just pass the
per-video one.
43     - The CLI `--output-dir` (already in `backend/main.py`) becomes the **parent**; the
pipeline
44     creates `<output-dir>/<video_slug>/` and writes everything under it.
45     - **DB choice (owner decision):** (a) a per-video `sports_indexer.db` in the slug dir
(simplest,
46     fully isolated, but loses cross-video queries), or (b) keep the shared DB and only
isolate
47     *files* (the slug dir holds artifacts; the DB stays one queryable store keyed by
`video_id`).
48     **Recommend (b)** ? the DB is already `video_id`-scoped; isolate the heavy *file*
artifacts,
49     keep one queryable metadata store. The slug dir then omits its own DB.
50     - **Back-compat:** default parent stays `output/`; a flag (or just the default-on
behaviour)
51     switches flat?per-video. Old flat outputs are left untouched; new runs go to slug
dirs.
52
53     ## Phasing (owner-gated; ONE PR per slice, off `master`)
54     - **L0 ? this plan.**
55     - **L1 ? `video_slug` helper + thread `run_dir`** through `trajectory_hybrid` (replace
the two
56     hardcoded `"output"` joins) + the per-video subdir creation; CLI `--output-dir` =
parent.
57     Pure plumbing; a regression test pinning that all artifacts land under the slug dir.

```

```

58 - **L2 ? runlog + report + rallies.csv into the slug dir**; migrate the eval/CSV
writers.
59 - **L3 ? cache-hygiene + cleanup_caches awareness** of the new layout (per-video purge).
60
61 ## Relation to the first isolated run
62 `output/GX010125_run/` (the 2026-06-14 offline re-score) is the manual prototype of
this
63 layout ? it isolated the rally CSVs + summary, but the trajectory still lives in the
shared
64 `output/wasb/` precisely because of the hardcoded path L1 fixes. This plan makes that
isolation
65 automatic and complete for every video.
66

```

## 5. user (2026-06-16T02:24:58.075Z)

```

1 # How rally detection works (the 2-layer mental model)
2
3 A short conceptual map of how we go from raw video to rally `[start, end]` segments ?
4 and, crucially, which layer our annotation can improve.
5
6 ...
7 video ??? frames ??? [DETECTION] ??? shuttle (x,y,visible) per frame
8             ?             ?
9             (WASB CNN,      per-frame trajectory
10             frozen weights) ?
11                               ?
12                               velocity signal (how fast the shuttle moves)
13                               ?
14                               [SEGMENTATION] ??? the 3 knobs we tune
15                               ?
16                               ?
17                               rally windows [start, end] ??? IoU vs golden
labels
18 ...
19
20 ## Two layers ? keep them separate in your head
21
22 **1. DETECTION ? "where is the shuttle, this frame?"**
23 WASB is a CNN that, for each frame, outputs a heatmap ? the shuttle's `(x, y)` + a
24 visibility/confidence. Its weights were learned (by gradient descent, on shuttle-
coordinate
25 labels) and we use them frozen. We chose a shuttle detector on purpose: "shuttle
in
26 flight" is the signal for "a rally is happening," and it's camera-agnostic ? unlike
27 player-motion, which fooled the old heuristic on broadcast cameras that pan with the
players.
28 (Your videos are static tripod, so player-motion would also work ? but shuttle-motion is
the
29 robust general bet.)
30
31 **2. SEGMENTATION ? "when is a rally?"**
32 From the per-frame `(x, y)` we compute the shuttle's velocity (displacement between
frames).
33 Then three knobs turn that into rally intervals:
34 - velocity_thresh ? a frame is "active" only if the shuttle is moving faster than
this
35 (in flight, not lying on the floor / being picked up).
36 - merge_gap ? active frames within this many seconds get joined into one rally
(bridges
37 brief gaps: a shuttle momentarily occluded, or the beat between two hits).
38 - min_window_duration ? throw away windows shorter than this (blips / false
flickers).
39
40 Output: a list of `[start, end]` windows = the rallies.

```

```

41
42     ## Where the annotation fits (the key point)
43     Our Roora/golden labels are **rally `[start, end]` times** ? they say *when* rallies
are, not
44     *where the shuttle is*. So they can't teach WASB's detector (that needs `(x,y)` labels).
They
45     **can** grade and tune the **segmentation layer**: we score predicted windows against
the golden
46     rallies with **temporal IoU** (how much they overlap), and pick the 3 knobs that
maximise it ?
47     **calibrate on some rallies, measure on held-out ones** so the number is honest.
48
49     ## Where sparks could land (open questions ? not commitments)
50     Things worth a shower-thought; any of these could beat 3 hand-tuned thresholds:
51     - **Richer features than raw velocity:** acceleration, trajectory *continuity* (a real
rally is a
52     smooth arc that survives occlusion), shuttle *height/arc*, or **direction reversals**
(each
53     reversal ? a shot ? ties to `shots_count`!).
54     - **A tiny learned segmenter** instead of 3 thresholds: feed the trajectory features to
a small
55     model trained on the golden rally labels (still the segmentation layer, just smarter).
56     - **Hysteresis** (two thresholds: enter-rally high, stay-in-rally low) ? robust to mid-
rally dips.
57     - **Rhythm/periodicity:** rallies have a hit cadence; dead time doesn't.
58     - **Fusion:** combine shuttle-motion (WASB) with player-motion for static cams. ?
designed in
59     [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md) (one rally-fusion layer;
the person cue is
60     ADR-007's parked #2 and `rally_gate.py`'s named-but-unbuilt "Gate B").
61     - **Per-video calibration as a *feature*:** "watch a few labeled rallies ? auto-tune to
this
62     court/lighting ? segment the rest sharply" ? exactly your earlier idea, at the right
layer.
63
64     The whole point of per-video calibration: a handful of *your* labeled rallies nudges
these knobs
65     to *your* court/camera, and we report the IoU lift on the rallies it never saw.
66

```

## 6. user (2026-06-16T02:24:58.503Z)

```

1     # Multi-signal rally fusion ? add a human (person) cue alongside the shuttle
2
3     **Status:** **P1 DONE** (2026-06-13) ? the person cue is extracted to
4     `backend/pipeline/detectors/person_detector.py` (`PersonDetector`), consumed by
`yolo_hybrid` with no
5     behavior change. **P2?P4 remain owner-gated.** Phases below are sequenced but each is a
separate PR.
6     **Owner decisions captured** in [S9](#9-open-decisions-for-the-owner).
7
8     > **Companion docs:** the 2-layer mental model is
[HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md);
9     > the modular-fusion architecture this builds on is **ADR-007**
([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md));
10    > the permissive-dependency rule is **ADR-002 / ADR-005**; the audio cue (the #1 next
cue, already
11    > scaffolded) is [archives/research/AUDIO_RALLY_DETECTION_PLAN.md](archives/research/AUD
IO_RALLY_DETECTION_PLAN.md).
12    > The vendor labels that gate the learned path come from
[GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md)
13    > and the labeling spec in [ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md).
14    > **How each cue here is added/tested without production regression** ? A/B, shadow
eval, and the
15    > promote-only-on-lift discipline ? is [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md)

```

```

(every P1?P4 cue
16     > below lands flag-gated, default OFF, promoted only on measured LOVO lift).
17
18     ---
19
20     ## 1. Context ? why this, why now
21
22     Rally detection today runs one cue at a time. Each segmenter independently turns a
single signal
23     into rally windows:
24
25     | Segmenter | Signal | File |
26     |---|---|---|
27     | `wasb_hybrid` / `tracknet_hybrid` | shuttle velocity (frozen WASB/TrackNet trajectory)
| `backend/pipeline/detectors/tracknet_runner.py` |
28     | `yolo_hybrid` (the Pydantic `default_segmenter` fallback; `config.json` overrides it
to `gemini`) | person velocity (torchvision + ByteTrack) |
`backend/pipeline/segmenters/yolo_hybrid.py` |
29     | `motion` | raw pixel motion | `backend/pipeline/segmenters/motion.py` |
30     | `gemini` | a cloud VLM watching the whole clip | `backend/providers/gemini.py` |
31
32     There is no fusion. The owner's directive: make the detector use the shuttle and
the humans
33     together ? and be ready to fold in more signals (audio, later pose) ? delivered
34     MIT / Apache-2.0 / BSD-clean so we can ramp commercially without a licensing
rethink.
35
36     This is not greenfield ? it operationalizes work the repo already blessed:
37
38     - HOW_RALLY_DETECTION_WORKS.md open question: "Fusion: combine shuttle-motion
(WASB) with
39     player-motion for static cams."
40     - ADR-007 already designed a modular "rally layer" that fuses cues outside the
frozen WASB
41     model, with audio as the #1 next cue and person/pose parked as #2. This plan picks
up the parked
42     person cue and generalizes the fusion seam so future cues are "register a producer,"
not "re-touch
43     the segmenter."
44     - backend/pipeline/detectors/rally_gate.py already exists ? the net-crossing gate,
explicitly
45     labelled in its own docstring as "Gate A in the over-fire fix spectrum (B = player-
stance state
46     machine, future)". The person cue in this plan is that named-but-unbuilt Gate
B.
47
48     ### The owner's "held-shuttle" false positive ? what's actually going on
49
50     The owner observed a "rally" that was just a player holding / flicking the shuttle in
hand between
51     points. This is the exact failure `rally_gate.py` was written to kill:
52
53     > "WASB over-detects: when a player flicks/tosses the shuttle back to the opponent for
service after
54     losing a point, that single trajectory looks like a rally. The discriminator is
structural: a real
55     rally has the shuttle crossing the net MULTIPLE times; a single service feed crosses
~once."
56
57     The structural fix already exists and is validated offline ? but it is NOT wired into
the production
58     runtime path. backend/pipeline/detectors/rally_gate.py is called only from the
eval drivers
59     (`eval_partial_labels.py`, `export_wasb_segments.py`, `calibrate_local.py`;
`distill_local.py` uses

```

```

60     `rally_gate.gate_windows` to compute the `net_crossings` feature, not `apply_gate`); the
live segmenters
61     historically didn't apply it. P2 closed this: `FusionConfig.min_crossings` now exists
(default `0` =
62     OFF), so the served Gate A is a config flag ? consistent with $10 P2 below, marked DONE.
So the
63     single cheapest, highest-confidence win in this whole plan is **P2's first step: fold
the existing
64     Gate A into production.** Everything else (the person cue / Gate B, the learned fusion)
is robustness
65     on top of that.
66
67     ### Why humans help ? and the honest risk
68
69     The beachhead videos are **static tripod** (Bangalore academies,
[PMF_MARKET_RESEARCH.md](PMF_MARKET_RESEARCH.md)),
70     so player presence/motion is a usable cue. ADR-001's warning was specifically about
**broadcast pan**
71     cameras, where player-motion fooled the old heuristic ? *that failure mode does not
apply to a fixed
72     court cam.* But we keep **the shuttle as the camera-invariant primary** so the general
bet stays robust
73     on other camera types. Humans then add:
74
75     - **Precision** ? reject shuttle-on-floor / warm-up knock-ups / held-shuttle feeds when
the court
76     isn't occupied by ?2 players in a play posture.
77     - **Recall** (via the learned classifier) ? features that help bridge shuttle
occlusion/blur gaps.
78
79     ---
80
81     ## 2. Core architecture ? one rally-fusion layer, cues are swappable producers
82
83     Formalize ADR-007's diagram. Every cue is a **black box that emits a time-aligned
signal** (per-frame
84     activity and/or per-window features); **fusion happens in *our* layer, never inside a
frozen model.**
85
86     ```
87     frames ?? WASB (frozen, MIT)      ?? shuttle trajectory ???
88     frames ?? person detector (ours)  ?? tracks + boxes ???????
89     audio ?? hit detector (ours)      ?? hit events ?????????????? RALLY LAYER (ours) ??
rallies
90     [frames ?? pose/stance (ours; parked #3) ?? stance ?????????? feature + decision fusion
91     ```
92
93     - **Shuttle = primary, camera-invariant.** Windows still **seed** from
94     `TrackNetRunner.trajectory_to_action_windows()` (`tracknet_runner.py:234`). Other cues
*enrich and
95     adjudicate* shuttle-seeded windows; they do **not** replace the shuttle seed. Person
runs only over
96     **shuttle-seeded candidate windows + padding**, not the whole video ? which bounds the
added cost.
97     - **A small `RallyCue` contract.** Each producer yields `(frame_idx ? features)` + per-
window stats.
98     Adding a cue = register a producer; the existing `SegmenterRegistry`
(`segmenters/base.py:35`) is the
99     template for the pattern. Keep the single-cue segmenters registered as fallbacks.
100
101     ---
102
103     ## 3. The two fusion modes (both ? per the owner decision)
104
105     ### 3.1 Decision-level gate (safe baseline, ships first, no labels needed)

```

```

106
107   ```
108   rally_active = shuttle_motion ? (?? persons in court zone ? recent person track)
109   ```
110
111   The `? recent-track` clause tolerates 1?2 frame person-detector misses so the AND-gate
can't cause a
112   recall cliff. This composes with the existing net-crossing Gate A:
113   ```
114   keep window ? net_crossings ? 2                (Gate A ? EXISTS in rally_gate.py, eval-
only today)
115   ? (?? in-court players ? recent track)    (Gate B ? the person cue, NEW)
116   ```
117
118   Gate A kills the service-feed / held-shuttle case structurally; Gate B adds court-
occupancy evidence
119   for the cases a single trajectory can still fool (e.g. a knock-up that does cross the
net). No training
120   data required ? this is the label-free fusion that ships before any golden labels exist.
121
122   ### 3.2 Feature-level (primary, the learned path)
123
124   The cues emit a small set of aggregated, scale/translation-invariant per-window
features ? NOT raw
125   coordinates. Raw pixel coordinates would memorize *this* court/camera and fail to
generalize off a
126   still-small labeled set; counts / ratios / zone-membership / two-sidedness transfer.
127
128   What already feeds the classifier (`distill_local.py:40`, `FEATURE_NAMES`):
129   `duration, peak_velocity, mean_velocity, active_ratio, net_crossings` ? note
net_crossings` is
130   already a feature (computed via `rally_gate.gate_windows`).
131
132   Person features to add (~4, deliberately few):
133
134   | Feature | Meaning | Rally signal |
135   |---|---|---|
136   | `players_in_court` | median count of persistent in-court tracks | ? 2 (singles) / 4
(doubles); 0?1 = dead time |
137   | `two_sided_motion` | are moving players on both net-halves? | rally = two-sided;
one person feeding = one-sided |
138   | `mean_player_motion` | aggregate normalized player velocity | rally = sustained;
resting/standing = low |
139   | `in_court_ratio` | fraction of frames with ?2 in-court players | track stability vs
flicker |
140   | `shuttle_player_colocation` | fraction of frames the shuttle sits inside/adjacent to a
person box (hand region) | held = high; in-flight = mostly free space ? the direct held-
shuttle discriminator |
141
142   These join the shuttle features in the candidate `stats` JSON
143   (`database.py::add_candidate_segment`) and the feature vector of the existing
distilled classifier ?
144   the tiny numpy-only L2 logistic regression in `backend/eval/classifier.py` (ADR-004),
NOT a new net.
145
146   It scores each candidate window `P(real rally)` and replaces the Gemini call at runtime.
147
148   What the model learns: the weights on those few features ? e.g. down-weight a
shuttle-motion
149   window with 0?1 in-court players / one-sided motion / high shuttle-colocation (warm-up,
floor shuttle,
150   held-shuttle feed); up-weight ?2 players, two-sided, sustained motion, shuttle in free
flight.
151
152   Small-N discipline (non-negotiable):

```

```

153     - Keep to **~4 person features** ? more features on a still-small labeled set overfit;
that's exactly
154     why the model stays a logistic regression, not a net.
155     - Use the **leave-one-video-out (LOVO)** protocol that `distill_local.py` **already
implements**: train
156     on the other videos' candidates, predict the held-out *video*, score vs its labels ?
never a
157     held-out *window* from the same video (windows in one video are correlated; that
flatters the score).
158     - Training labels today are **Gemini silver labels** (distilled offline, ADR-004/007 ?
Gemini is a
159     teacher, never a runtime dependency, never a training-data *source* for owned
weights); **golden
160     labels are the honest measuring stick**, and as the Roora golden corpus grows they can
also become
161     training labels. Until the labeled set is larger, treat the learned weights as
162     **calibration / direction-check, not "a fully general model"*** ? the decision-level
gate (§3.1) is
163     the shippable, label-free fusion in the meantime.
164
165     > *(Gated on golden labels existing ? the same blocker ADR-007 flags for audio. See
166     > [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md).)*
167
168     ---
169
170     ## 4. Shuttle-state: in-flight vs in-hand (the misclassification, mapped to real code)
171
172     The "shuttle moved" seed conflates a shuttle **in flight** with a shuttle **carried in
hand**. The
173     discriminators, and where each lives:
174
175     | Signal | In-flight | Held / static | Status |
176     |---|---|---|---|
177     | `net_crossings` | ? 1 (usually several) | ~0 (held) / ~1 (single feed) | **EXISTS**
(`rally_gate.py`); eval-only ? wire to prod |
178     | `shuttle_player_colocation` | low (free space) | high (near a hand) | **NEW** (needs
the person cue) |
179     | `direction_reversals` | several (racket contacts) | ~0 | derivable from trajectory;
partial overlap with crossings |
180     | `peak shuttle speed` | high | walking speed | derivable from existing velocity |
181
182     So the in-flight-vs-held fix is **mostly Gate A (already built) + the new colocation
feature from the
183     person cue**. **No new label column is needed** ? the golden labels already start a
rally at the
184     *serve-contact frame* and treat all pre-serve / held-shuttle time as dead time (see
185     [ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md)), so held-shuttle moments
are negatives
186     by construction and are the ground truth that proves the detector wrong.
187
188     ---
189
190     ## 5. Adversarial mitigations (integrating safely)
191
192     - **Court-region mask** ? polygon around the court; drop persons outside it (spectators,
coaches,
193     adjacent courts). Config-supplied per camera; ~1 ms. Source: a manual per-camera
polygon first;
194     the **4 court corners + net-line** that Roora labels once per video (see the labeling
doc) build
195     this mask **and** the net split that `two_sided_motion` / `net_crossings` need.
196     - **Player-count constraint** ? expect **2 (singles) / 4 (doubles)** persistent in-court
tracks;
197     suppress transient/extra detections. The `singles_or_doubles` manifest flag sets the
expectation.

```

```

198 - Temporal association ? reuse `supervision` ByteTrack (already in
`yolo_hybrid`); reject
199 singleton detections; smooth across frames.
200 - Multi-court rejection ? proximity to the primary court region (homography
optional, later).
201 - Domain shift (COCO frontal ? side/overhead court) ? start box-only (robust);
keep the
202 detector swappable. Fine-tuning a person detector is parked ? we have no in-house
person labels,
203 and the paid-LLM/training guardrail + minors-exclusion apply to any future person-
training data.
204 - Fusion-degradation guard ? shuttle stays primary; person is gate + feature, never
the sole
205 trigger; the single-cue segmenters remain registered fallbacks.
206
207 ---
208
209 ## 6. Box vs pose
210
211 Box-first. Occupancy + count + two-sided motion + shuttle-colocation is enough for
both the gate
212 and the features, and box detection is the cheap, robust win. 2D pose is parked behind
the same
213 `RallyCue` interface ? pose (MediaPipe Pose / RTMPose, both Apache-2.0) buys serve-
ready stance
214 gating and shot cues later, at the cost of a second model. This matches ADR-007 parking
pose as the
215 #2/#3 cue.
216
217 ---
218
219 ## 7. Performance
220
221 The person detector is the existing in-process torchvision model run at frame-skip
375
222 (`yolo_frame_skip` default 3) with track interpolation between sampled frames. The
shuttle keeps
223 running in its separate WSL process ? the two are separate processes, so a "shared
backbone" isn't
224 free; budget realistically as the existing `yolo_hybrid` cost minus skip savings. Person
runs only
225 over shuttle-seeded windows + `ai_handoff_padding`, not the whole video, which bounds
the cost.
226
227 ---
228
229 ## 8. Licensing (the guardrail table ? all green)
230
231 Every dependency stays MIT / Apache-2.0 / BSD (ADR-002 / ADR-005; `ultralitics` was
dropped for
232 AGPL-3.0).
233
234 | Cue / model | Code | Weights / data | Verdict |
235 |---|---|---|---|
236 | Shuttle: WASB-SBDT, TrackNetV3 | MIT | frozen badminton weights | ? (ADR-001/002) |
237 | Person (default): torchvision Faster R-CNN / RetinaNet / FCOS | BSD/MIT | COCO-
pretrained | ? already in repo (`_DETECTOR_FACTORIES`) |
238 | Person (escalation): YOLOX, RT-DETR / RT-DETRv2 | Apache-2.0 | COCO /
Objects365 | ? |
239 | Pose (parked): MediaPipe Pose, RTMPose / MMPose | Apache-2.0 | COCO-structure
| ? |
240 | Tracking: `supervision` ByteTrack | MIT | n/a | ? already in repo |
241 | Banned | Ultralytics YOLOv5/8/11 (AGPL-3.0) . MonoTrack (Adobe, non-comm)
. YOLO-NAS (Deci, non-comm) | ? | ? |
242

```

```

243     **COCO caveat to record:** COCO *annotations* are **CC BY 4.0** (commercial OK *with
attribution*);
244     COCO *images* are Flickr-ToS (per-image). Pretrained-weight use is clean; record the
attribution.
245
246     ---
247
248     ## 9. Open decisions (for the owner)
249
250     1. Court-region mask source ? manual per-camera polygon (simple, ship first) vs auto
court
251         homography/line detection (later). Recommend manual first, fed by Roora's court-
corner labels.
252     2. Person detector default ? keep torchvision (zero new deps, BSD/MIT) vs adopt
YOLOX/RT-DETR
253         (Apache, faster) now. Recommend torchvision first, escalate only if eval shows a
speed/accuracy gap.
254     3. Pose now or parked ? recommend parked behind the interface (box-first).
255     4. Golden labels ? feature-level fusion (P3) is blocked on golden rally labels;
the source is
256         the VLC `rally-annotator` + the Roora vendor pilot
([GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md)).
257
258     ---
259
260     ## 10. Phased execution (owner-gated; ONE PR per slice, off `master`)
261
262     - P0 ? this design doc* +
[ROORA_LABELING_GUIDELINES.md](ROORA_LABELING_GUIDELINES.md), indexed in
263         `docs/README.md`, cross-linked from `HOW_RALLY_DETECTION_WORKS.md` and ADR-007. *(this
deliverable;
264         docs-only, no code.)*
265     - P1 ? extract the shared person cue:** ? DONE (2026-06-13).** Lifted the
torchvision detector +
266         ByteTrack + velocity out of `yolo_hybrid` into
`backend/pipeline/detectors/person_detector.py`
267         (`PersonDetector`); `yolo_hybrid` now orchestrates only and delegates to `self.person`
(no behavior
268         change ? regression-safe, proven by the unchanged process_video output tests). New
269         `tests/test_person_detector.py`; suite +1 green; person_detector is type-clean (left
mypy quarantine
270         for the moved code). The fusion segmenter (P2) reuses this same producer over shuttle-
seeded windows.
271     - P2 ? the FusionSegmenter (Gate A + Gate B + person-feature persistence):** ? DONE
(2026-06-14).**
272         Shipped `backend/pipeline/segmenters/fusion_hybrid.py::FusionSegmenter` (registry
`fusion_hybrid`,
273         default-OFF ? `default_segmenter` unchanged), subclassing the trajectory engine
via two identity
274         hooks (`_enrich_candidate_stats`, `_select_for_handoff` ? wasb/tracknet stay byte-
identical, adversarially
275         verified). It seeds windows from the shuttle substrate, always persists
`net_crossings`** (Gate A signal),
276         and ? when `indexing.fusion.cue_flags.person` is on ? runs the person cue over each
window's frames
277         (`PersonDetector.detections_per_frame`) and persists the ~5 person features
(`fusion_features.py`,
278         pure + unit-tested). Served gating is optional + default-OFF: `fusion.min_crossings`
(Gate A) /
279         `fusion.min_players` (Gate B) skip sub-threshold windows from the handoff while
persisting the full
280         superset** (offline substrate intact). Court mask = optional `fusion.court_polygon`
(None ? count-only).
281         Lift to be measured via the harness; held-shuttle-drop + person-feature values are
GPU/owner-side.

```

```

282     - **P3 ? feature-level (learned) fusion:** add the person features (incl.
`shuttle_player_colocation`)
283     to `distill_local` / `classifier`; recalibrate IoU on the golden set; report **LOVO
lift on held-out
284     videos**. *(Gated on golden labels.)*
285     - **P4 ? additional cues:** wire **audio** (ADR-007's #1, already scaffolded in
286     `backend/pipeline/audio/hit_detector.py`) and later **pose**, through the same
`RallyCue` interface.
287     Each is add-a-producer, not a refactor.
288
289     ---
290
291     ## 11. Verification
292
293     - **Offline, runnable in this dev container (no GPU/torch/cv2):** unit tests for the
fusion gate logic
294     and `RallyCue` aggregation (court-mask filter, player-count constraint, `? recent-
track` tolerance)
295     and the shuttle-state signals (net-crossing count ? extends the existing `rally_gate`
tests ? and
296     person-colocation) with **synthetic trajectories + person boxes**. A key-license
assertion test
297     (no `ultralytics` import; detector factories stay torchvision/Apache).
298     - **Machine-side (owner, GPU/WSL ? the full pipeline can't run in this container):** run
the
299     `FusionSegmenter` on a real static-tripod clip; confirm **held-shuttle / warm-up /
spectator false
300     positives drop** vs shuttle-only (this should reproduce the owner's observed bug and
show it fixed ?
301     start by confirming Gate-A-in-production alone fixes it); **LOVO IoU lift on the
golden set** for the
302     learned path.
303     - **Docs (P0):** links resolve; both docs indexed in `docs/README.md`; this doc cross-
linked from
304     `HOW_RALLY_DETECTION_WORKS.md` (the fusion open-question) and ADR-007.
305

```

## 7. assistant (2026-06-16T02:25:03.452Z)

Let me continue reading the rest of CODE\_MAP.md (detectors, data contracts, database) and the key code files in parallel.

## 8. user (2026-06-16T02:25:04.838Z)

File content (25560 tokens) exceeds maximum allowed tokens (25000). Use offset and limit parameters to read specific portions of the file, or search for specific content instead of reading the whole file.

## 9. user (2026-06-16T02:25:05.365Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6     1. Translate the Windows video path to its WSL-visible form (and optionally
7     stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8     2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9     3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10    4. Convert that trajectory into high-motion "action windows" ? the same
11    candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12    pipeline (AI handoff, DB population) is unchanged.
13
14    Nothing here imports TensorFlow; all model code stays in WSL.

```

```

15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"           # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5
39         stage_in_wsl: bool = True                        # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                  # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                          # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                       # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                        # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p

```

```

72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "wsl not found ? is this running on Windows with WSL2 installed?"
112         except subprocess.TimeoutExpired:
113             return False, "WSL healthcheck timed out."
114         out = (res.stdout or "").strip()
115         if res.returncode != 0:
116             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
117         return True, out
118
119 # ----- inference
120
121 def run_predict(self, video_win_path: str, output_win_dir: str,
122                 video_id: Optional[str] = None) -> Optional[str]:
123     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
124     None.
125
126     ``output_win_dir`` is a Windows directory (under the repo's output/) that
127     WSL writes into via its /mnt view, so the Windows process can read the CSV
128     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
129     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
130     a filename cannot collide on the output CSV.
131     """
132     if not self.cfg.weights_path:
133         logger.error(
134             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
135             "in config.json to the WSL path of your trained TrackNetV4 weights."

```

```

134         )
135         return None
136
137     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138     # relative paths through unchanged, so WSL would resolve them against the
139     # predict.py cwd instead of the repo (same bug class as wasb_runner).
140     output_win_dir = os.path.abspath(output_win_dir)
141     os.makedirs(output_win_dir, exist_ok=True)
142     # Normalize separators so basename/splitext work when tests (or collector)
143     # pass windows-style paths on a Linux host.
144     video_win_path = str(video_win_path).replace("\\", "/")
145     base = os.path.basename(video_win_path)
146     stem, ext = os.path.splitext(base)
147
148     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149     if ext.lower() not in (".mp4", ".avi"):
150         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151         return None
152
153     wsl_out = to_wsl_mnt_path(output_win_dir)
154     # Namespace by video_id so two same-filename videos don't collide on the CSV.
155     # predict.py derives its output name from the (staged) video stem, so staging
156     # under the namespaced name propagates into "<key>_predict.csv".
157     key = f"{stem}__{video_id[:12]}" if video_id else stem
158     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         # "<stem>_predict.csv".
169         # We cannot rename its output, so fall back to the un-namespaced name.
170         wsl_video = to_wsl_mnt_path(video_win_path)
171         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173     if self.cfg.min_free_gb > 0:
174         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175         if free is not None and free < self.cfg.min_free_gb:
176             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                            "consider running tools/cleanup_caches.py.",
178                            free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180     cmd = (
181         f"conda activate {self.cfg.conda_env} && "
182         f"cd {self.cfg.repo_dir}/src && "
183         f"python predict.py "
184         f"--video_path {wsl_tilde_quote(wsl_video)} "
185         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186         f"--output_dir {wsl_tilde_quote(wsl_out)} "
187         f"--queue_length {self.cfg.queue_length}"
188     )
189     logger.info("Running TrackNet inference on %s ...", base)
190     try:
191         res = self._wsl_bash(cmd)
192     except subprocess.TimeoutExpired:
193         logger.error("TrackNet inference timed out after %ss.",
194                     self.cfg.timeout_sec)
195         return None
196
197     if res.returncode != 0:
198         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or

```

```

res.stdout)[-2000:])
197         return None
198
199         if not os.path.exists(expected_csv_win):
200             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201             return None
202             logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204             # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205             # regenerable intermediate). On earlier failure we return above without
cleaning.
206             if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207                 logger.info("Cache hygiene: removing staged video for %s "
208                             "(set indexing.tracknet.keep_frames=true to retain).", key)
209                 self._wsl_rm_rf([wsl_video])
210             return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224         try:
225             res = self._wsl_bash(cmd)
226         except subprocess.TimeoutExpired:
227             logger.error("Staging video into WSL timed out.")
228             return None
229         if res.returncode != 0 or "OK" not in (res.stdout or ""):
230             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231             return None
232             return dest
233
234     # ----- CSV -> windows
235
236     @staticmethod
237     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239         points: List[TrajectoryPoint] = []
240         with open(csv_win_path, newline="") as f:
241             reader = csv.DictReader(f)
242             for row in reader:
243                 try:
244                     points.append(TrajectoryPoint(
245                         frame=int(row["Frame"]),
246                         visible=int(row["Visibility"]) == 1,
247                         x=float(row["X"]),
248                         y=float(row["Y"]),
249                     ))
250                 except (KeyError, ValueError):
251                     continue
252             points.sort(key=lambda p: p.frame)
253             return points
254
255     @staticmethod
256     def trajectory_to_action_windows(

```

```

257     points: List[TrajectoryPoint],
258     fps: float,
259     frame_width: float,
260     chunk_start: float = 0.0,
261     velocity_thresh: float = 0.02,
262     merge_gap: float = 2.0,
263     min_window_duration: float = 2.0,
264     chunk_index: Optional[int] = None,
265 ) -> List[Dict[str, Any]]:
266     """Convert a shuttle trajectory into candidate rally windows.
267
268     A frame is "active" when the shuttle is visible AND its inter-frame
269     displacement (normalised by frame width, per second) exceeds
270     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
271     ``merge_gap`` seconds of each other are grouped into one window; short
272     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
273     shape feeding the AI-handoff phase is identical.
274     """
275     if fps <= 0:
276         fps = 30.0
277     if frame_width <= 0:
278         frame_width = 1280.0
279
280     # Compute per-frame normalised velocity from the last *visible* point.
281     active = [] # (frame_idx, velocity)
282     prev: Optional[TrajectoryPoint] = None
283     for p in points:
284         if not p.visible:
285             prev = None # break the trajectory; absent shuttle = not in flight
286             continue
287         if prev is not None:
288             dframes = p.frame - prev.frame
289             if dframes > 0:
290                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
291                 velocity = (dist / frame_width) * (fps / dframes)
292                 if velocity > velocity_thresh:
293                     active.append((p.frame, velocity))
294             prev = p
295
296     if not active:
297         return []
298
299     max_gap_frames = int(merge_gap * fps)
300
301     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
302         vels = [v for _, v in group]
303         return {
304             "start": group[0][0] / fps + chunk_start,
305             "end": group[-1][0] / fps + chunk_start,
306             "active_frame_count": len(group),
307             "peak_velocity": max(vels),
308             "mean_velocity": sum(vels) / len(vels),
309             "track_id_count": 1, # single shuttle; kept for window-shape parity
310             "chunk_index": chunk_index,
311         }
312
313     windows = []
314     group = [active[0]]
315     for af in active[1:]:
316         if af[0] - group[-1][0] <= max_gap_frames:
317             group.append(af)
318         else:
319             windows.append(_finalize(group))
320             group = [af]
321     windows.append(_finalize(group))

```

```
322
323         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
324
```

## 10. user (2026-06-16T02:25:05.843Z)

```
1      """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3      This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4      of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5      the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7      The ABC lets us:
8      - Keep the current WSL adapters working unchanged for Windows users.
9      - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10     the hybrid segmenters or the model-agnostic windowing brain.
11     - Provide a single place to evolve the healthcheck / error contract and thread
12     detector readiness into per-video reports.
13
14     Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15     - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16       Run detector, return the Windows-side path to the produced trajectory CSV
17       (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18       The caller (hybrid) is responsible for output dir; impls may stage the video
19       into WSL fs for perf.
20
21     - healthcheck() -> tuple[bool, str]
22       Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23       Message is logged (and can be surfaced to reports or UX in future).
24       Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26     Existing behavior is 100% preserved. The only additions are the inheritance and
27     the explicit common type for future implementers.
28
29     How to add a new runner (for docs / future contributors):
30     1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31     2. Implement run_predict and healthcheck (return (bool, str) tuple).
32     3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33       TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34       deliberately static and model-agnostic). If the new detector needs different
35       windowing, it can implement its own or we generalize later.
36     4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37       registry never pulls torch/tf on a machine that doesn't have them).
38     5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39       DetectorRunner instances (they already do the right thing via duck-typing + the
40       type hints we add here).
41     6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42       returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43     7. No change to eval/calibration code paths (they consume the CSV + the static
44       windowing helpers directly).
45
46     Optional Linux-native path (the main de-risk goal):
47     A LinuxNativeRunner (or ONNXRunner) would:
48     - Skip all wsl / _stage / conda activate.
49     - Load weights locally (torch or onnx) or call a native binary.
50     - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51     - healthcheck can just check "weights file exists and torch importable + CUDA?".
52     This removes the WSL requirement for Linux users / future SaaS workers while
53     keeping the exact same candidate quality characteristics (or better, once native
54     weights are available).
55
56     Error surfacing into reports:
57     The healthcheck message is already logged by the hybrids on failure. The reporting
58     harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59     segmenter Results (or pass detector_health into emit_indexer_report context), the
```

```

60 (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61 stage. The ABC makes that future change local to one place.
62
63 See also:
64 - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65 - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66 - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67 """
68
69 import logging
70 import shlex
71 import subprocess
72 from abc import ABC, abstractmethod
73 from typing import Any, Callable, List, Optional, Tuple
74
75 logger = logging.getLogger(__name__)
76
77
78 def wsl_tilde_quote(path: str) -> str:
79     """shlex.quote that still lets bash expand a leading `~/`.
80
81     ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
82     treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
83     directory" ? this broke live WSL staging for the default ``~/clips``
84     stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
85     "shlex-quote breaks ~ expansion" item). Quoting only the part after
86     ``~/`` keeps expansion AND protects spaces/specials in the remainder.
87     """
88     if path == "~":
89         return path
90     if path.startswith("~/"):
91         return "~/ " + shlex.quote(path[2:])
92     return shlex.quote(path)
93
94
95 class WslCommandMixin:
96     """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.
97
98     Lives beside (not on) DetectorRunner: a future Linux-native runner must not
99     inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
100     and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
101     ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
102     """
103
104     cfg: Any
105
106     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
107         full = f"source {self.cfg.conda_sh} && {inner_cmd}"
108         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
109         logger.debug("WSL exec: %s", full)
110         return subprocess.run(argv, capture_output=True, text=True,
111                               timeout=(self.cfg.timeout_sec or None))
112
113     # ---- cache hygiene helpers (shared by both WSL runners) -----
114     def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
115         """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.
116
117         Returns None if it can't be determined (never raises). Used as a pre-run
118         disk guard so a long extraction doesn't fill the disk mid-flight.
119         """
120         try:
121             res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
122             'NR==2{{print $4}}'")
123             except (subprocess.TimeoutExpired, OSError):
124                 return None

```

```

124         try:
125             return int((res.stdout or "").strip()) / (1024 ** 3)
126         except (ValueError, AttributeError):
127             return None
128
129     def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
130         """Best-effort recursive delete of WSL paths (post-success cleanup; never
131         raises)."""
132         paths = [p for p in wsl_paths if p]
133         if not paths:
134             return
135         quoted = " ".join(wsl_tilde_quote(p) for p in paths)
136         try:
137             self._wsl_bash(f"rm -rf {quoted}")
138         except (subprocess.TimeoutExpired, OSError) as e:
139             logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
140
141     def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
142         """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.
143
144         Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
145         filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
146         fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
147         trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
148         A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
149         any
150         frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
151         ``cp`` still gets its chance (we never block a genuinely-readable source).
152         """
153         try:
154             res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
155         except (subprocess.TimeoutExpired, OSError):
156             return None
157         if "READABLE" in (res.stdout or ""):
158             return None
159         return (
160             f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
161             f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
162             f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
163         )
164
165     class DetectorRunner(ABC):
166         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
167
168         @abstractmethod
169         def run_predict(self, video_win_path: str, output_win_dir: str,
170                        video_id: Optional[str] = None) -> Optional[str]:
171             """Run inference and return path to trajectory CSV (or None on failure).
172
173             ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
174             videos that happen to share a filename cannot reuse each other's frames or CSV.
175             It is optional for back-compat; callers in the live pipeline always pass it.
176             """
177             pass
178
179         @abstractmethod
180         def healthcheck(self) -> Tuple[bool, str]:
181             """Quick check if the runner (env, weights, etc.) is usable.
182
183             Returns (ok, message). Never raises. Cheap to call before a long run.
184             """
185             pass
186
187     # De facto trajectory contract: the trajectory hybrids call these

```

```

187         # unconditionally after run_predict. Concrete runners reuse/re-export the
188         # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
189         # a runner with custom windowing supplies its own with the same shapes.
190         parse_trajectory_csv: Callable[..., Any]
191         trajectory_to_action_windows: Callable[..., Any]
192

```

## 11. assistant (2026-06-16T02:25:10.264Z)

Now I have the trajectory CSV contract and windowing. Let me read the database persistence, the trajectory\_hybrid engine, wasb\_runner, rally\_gate, fusion\_features, the stitcher compiler, and the remaining CODE\_MAP data-contract sections.

## 12. user (2026-06-16T02:25:12.039Z)

```

152         - `::TrackNetConfig`(+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro`
field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
153         - `::to_wsl_mnt_path` (`C:\x`->`\mnt/c/x`; ? POSIX/\`~` passthrough, UNC unhandled),
`::TrajectoryPoint`(frame,visible,x,y).
154         - `::parse_trajectory_csv` @staticmethod** + `::trajectory_to_action_windows`
@staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names
`<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`,
`frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout
(hangs forever). `::stage_video` "OK" sentinel.
155         - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner)
(+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default
`wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::INFER_WIN`) +
video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` =
`staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-
named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing windows copy inert until
staged); `wsl_bash` now shared via `detectors/base.py::WslCommandMixin` (2026-06-11 dedupe).
Both runners now satisfy the DetectorRunner ABC for future swappability. **Cache hygiene
(#109):** on a SUCCESSFUL run both runners delete their WSL intermediates ? WASB the frame cache
+ staged video, TrackNet the staged video ? gated by `indexing.{wasb,tracknet}.keep_frames`
(default false; on FAILURE kept so a re-run resumes). Pre-run `min_free_gb` disk-guard warning.
Shared helpers `WslCommandMixin::wsl_rm_rf`/`wsl_free_gb`.
156         - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT
repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache:
`::DET_FILE='det_raw.jsonl'`, `::MANIFEST_FILE='manifest.jsonl'`, `::CACHE_VERSION=1`,
`::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to
heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights`
compared raw), `::atomic_write_text`/`atomic_write_trajectory`, `::dets_to_tracker` (xy MUST be
np.array), `::build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW),
`::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker
STATEFUL -> always fully re-run; `--fresh` ignores cache.
157         - `@backend/pipeline/detectors/person_detector.py::PersonDetector` ? the **PERSON rally
cue** (torchvision detector + supervision ByteTrack + velocity windowing), extracted from
yolo_hybrid (fusion P1; type-clean, NOT quarantined). `::PERSON_CLASS_ID=1` ? (ultralytics used
0; wrong value -> zero detections),
`::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. `::lazy_load_model`
(lazy `torchvision` import so module/tests load without it; `self.model` typed `Any`),
`::make_tracker` (per-chunk ByteTrack ? IDs chunk-local), `::detect_and_track` (one frame ->
confident persons w/ track IDs; BGR->RGB, COCO person filter, ByteTrack association),
`::extract_action_windows_from_chunk` (chunk -> high-motion windows w/ `peak/mean_velocity`,
`active_frame_count`, `track_id_count` ? the stats that feed candidate `stats` AND the future
learned-fusion person features). ? velocity at `effective_fps = fps/frame_skip` ->
`yolo_velocity_threshold` coupled to `frame_skip`. Consumed by yolo_hybrid AND the fusion
segmenter (P2 reuses the SAME producer over shuttle-seeded windows via `::detections_per_frame`
? add-a-producer, not a refactor). BSD torchvision + MIT ByteTrack.
**`::detections_per_frame(video_path,start_frame,end_frame,frame_skip)`** (fusion P2) reads the
ORIGINAL video by ABSOLUTE frame range (seek+sequential) ? `{frame_idx: [(track_id,bbox)]}` +
`fps/frame_width/frame_height`, so indices align 1:1 with the WASB trajectory `.frame` (no clip
re-timing). ? cv2 frame-seek accuracy varies by codec ( $\pm$  a few frames on real footage).
158         - `@backend/pipeline/detectors/fusion_features.py` ? **pure** per-window person-cue

```

```

feature math (no I/O/torch/cv2; mirrors rally_gate discipline). Inputs: per-frame
`{frame_idx:[(tid,bbox)]}` + the in-window shuttle points + optional normalized `court_polygon`
+ `net=(axis,pos)`. `::point_in_polygon` (ray-cast), `::person_in_court` (foot-point; None
polygon=count-all; bad dims=False), `::in_court_counts`, `::players_in_court` (median),
`::in_court_ratio`, `::mean_player_motion` (per-track centre displacement; x by width, y by
height), `::two_sided_motion` (both net-halves occupied), `::shuttle_player_colocation` (shuttle
inside a person box  $\pm$  `pad_frac` halo over aligned visible frames ? **the held-shuttle
discriminator**), `::compute_person_features` (aggregator ? the 5 `::PERSON_FEATURE_NAMES`). All
scale/translation-invariant, graceful to None/empty. Tests: `tests/test_fusion_features.py`.
159 - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure,
no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'),
`::count_net_crossings` (sign changes vs median net; deadband jitter zone),
`::gate_windows` / `::apply_gate` (`min_crossings=2` default; kills single-toss service feeds;
**estimate=`param` lets a caller scoring many windows pass a precomputed `(axis,net,span)` to
skip per-window re-estimation ? used by FusionSegmenter).
`::GatedWindow` (start,end,crossings,visible_points,kept). ? window contract here is
(start,end) tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local /
distill_local / export_wasb_segments / fusion_hybrid.
160
161 ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
162 - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`,
`::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape),
`::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN)
for mean_iou/boundary errors.
163 - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end`
CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`::write_scaffold`.
164 - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video
k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video;
needs ? 2 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT),
`::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. $
`PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap
>=0.1` = overfit alarm.
165 - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfigs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` ($`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
166 - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing +
net-crossing gate, no cloud at runtime). `::local_grid` (180 cfigs), `::make_local_predict_fn`
(adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a
labels file yields no rallies), `::run`/`::main`. $
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
167 - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison
+ optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video),
`::SEG_FIELDS`, `::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::maybe_slice`
(lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads
straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is
load-bearing.
168 - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in
run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1),
`::resolve_stages_for` (`auto`; `::FINAL_ONLY_SEGMENTERS={motion,gemini}`),
`::resolve_video_path` (normalizes collector `./data\..` windows paths), `::format_aggregate`.
169 - `@backend/eval/harness.py` ? DB-pred scorer (no re-run).
`::STAGES=('candidates','final')`, `::get_predictions`
(candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`; ? unknown
stage raises ValueError; reused by regression.py),
`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate), `::format_report_text`.
170 - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default
0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn` is the substrate plug-in seam.

```

```

171 - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid`
clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON "model":"logreg" tag.
172 - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels**
(PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail
detections aren't FPS ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops
table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid
vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ?
`window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ?
P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate
+ metrics.
173 - `@backend/features/rally_seq.py` ? **THE single source of truth for the Gen-0
rally_seq per-frame feature schema** (ADR-008): training (`training/`) AND the serving/eval path
import the SAME `build_frame_arrays`/`features` so train==serve by construction. Pure
numpy/scipy (no torch/cv2). `::FEATURE_SCHEMA_VERSION` (=1; bump on ANY semantic change ?
artifacts pin it, loader hard-fails on mismatch), `::FEAT_NAMES` (the 6:
`vis_rate,spd_mean,spd_max,cross_rate,x_std,y_std`), `::FEATURE_SCHEMA`
(`win_sec`/`stride_sec`/`fps_semantics`/`occlusion_rule`),
`::build_frame_arrays(points,frame_width,fps)` (per-frame vis/x/y + per-visible-frame speed &
net-crossings; speed normalized per-SECOND `*fps`, NOT per-frame ? the 2026-06-10 fps-skew fix;
skips speed/crossings across occlusion gaps > `max(2,round(0.25*fps))`),
`::features(arr,fps,win_sec,stride_sec)` (scipy `uniform/maximum_filter1d` windowed -> `(X[n,6],
times)`). ? re-exported by `@backend/eval/rally_seq_proto.py`.
174 - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter =
owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean,
spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO. ?
`build_frame_arrays`/`features` (+`FEAT_NAMES`) are NOT defined here ? they are **re-exported**
(`noqa F401`) from `@backend/features/rally_seq.py` (ADR-008 single-sourcing; speed per-SECOND +
occlusion-gap-guarded since the 2026-06-10 sweep). Locally-owned: `::labels_for`,
`::probs_to_segments` (smooth->threshold->merge/min-dur), `::roc_auc` (rank-based with **midrank
ties**, scipy.stats.rankdata), `::run` (per-video AUC + LOVO-threshold F1 + oracle-threshold
F1). ? DIRECTIONAL only (n=6, linear, threshold-transfer unsolved); held-out per-frame AUC
0.83?0.92, **beats the heuristic on multi-court footage** (Kushagra/gBaaddy). **n=6 LOVO:
learned 0.626 vs heuristic 0.480** (learned rose from 0.583 after the fps-normalization fix).
Manifest `output/human_lovo_manifest.json` (n=6; trajectory paths now under `Annotation
Setup/Collect/Trajectories/`) shared w/ calibrate_local.
175 - ? **MULTI-COURT / background-game confound (2026-06-08 finding):** WASB tracks EVERY
shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high
dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't
find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally÷dead-
time visibility "contrast" (?3.6× heuristic works, ?1.9× fails). Lever = **foreground-court
isolation** (spatial gate to the prominent court, needs real court-region detection). Academy =
multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
176 - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond
calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT:
duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal),
`::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`),
`::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-
one-video-out needs ?2 videos; final model trained on all.
177 - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT
padded clips (precision lever; eval/tuning, NOT the live pipeline).
`::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (?
FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`,
`::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises
SystemExit); imports `BASELINE` from calibrate_wasb +
`TUNED`/`write_segments_csv`/`seconds_to_mms` from export_wasb_segments.
178 - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=eval_baselines/regression_baseline.json` (tracked, NOT under gitignored
output/), `::regress`, `::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;
"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle),
`::save_baseline`/`::load_baselines`, `::gt_hash` (label-set fingerprint, reused by
experiment.py), `::RegressionResult`. ? imports `annotation_registry` from
**`backend.annotations`** NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass). CLI

```

```

`run_regression.py` exit 0 ok / 1 regress / 2 no-DB / 3 no-baseline = the promotion gate.
179 - `@backend/eval/experiment.py` ? A/B + shadow + **SxS** variant layer (the thin
"experiment" glue per `docs/EXPERIMENT_HARNESS.md`; composes
metrics/calibration/classifier/training/regression ? NO new scoring math).
`::Variant`(segmenter,config_overrides,cue_flags,`min_crossings`[Gate A],`min_players`[Gate
B],classifier_model,feature_names,threshold,merge_gap;
`to_dict/from_dict/spec_hash/is_learned`), `::CONTROL` (pinned zero-gate baseline).
`::VideoCandidates`(name,intervals,features{col-major},gts), `::predict` (gate?learned-
filter?`gemini_refine.merge_overlaps`), `::evaluate_variant` (static / pinned-model / **honest-
LOVO** train-on-others ? mirrors distill_local), `::compare` (labeled **B?A** deltas +
`label_set_hash`), `::compare_unlabeled` (**NEW-footage SxS**: agreed/a_only/b_only via
`match_intervals`, no GT ? the owner's "SxS on new videos"), `::record_experiment` (?
`eval_baselines/experiment_leaderboard.json`), `::load_video_candidates` (DB bridge via
`query_candidate_segments`). ? enabling a gate needs its feature persisted (absent?0.0?fails
gate, warns); learned variant on an unlabeled video needs a pinned `classifier_model`. **No-reg
invariant (tested): all cues OFF ? byte-identical to CONTROL.**
180
181 ### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
182 - `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure
numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent`(t,strength),
`::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy
STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k.std`;
`k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2x, band-pass null -> NOT adopted
(PR #35).
183 - `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits`>=2 so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
184
185 ### 2.4 Stitcher / tools / utils
186 - `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`/`end_padding` ctor defaults 0.5/1.0 but
every live call site (`/api/compile`, CLI trim, `scripts/run_process_and_stitch.py`) threads
`config.json` stitching.{begin_padding,end_padding} (typed default 0.5/0.5); temp clips in a
TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break. ? optional **branding
watermark** (`stitching.watermark`, default-on): `::watermark_filter` builds a `-vf` (drawtext
via `textfile`= + optional logo `movie/overlay`) applied to the per-clip re-encode only (concat
stays `-c copy`); a failed watermark encode retries the clip WITHOUT it (never nukes the reel,
and logs the ffmpeg stderr reason). Pass `watermark=` to `VideoCompiler(...)` (a
`WatermarkConfig` model or dict; `None` = off). Default font = bundled Apache-2.0
`::BUNDLED_FONT` (`backend/assets/fonts/RobotoSlab-Regular.ttf`) so it renders without
fontconfig; `font_path` overrides.
187 - `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure,
unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow`(rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `_read_time` -> backend/eval/annotations.parse_timestamp.
188 - `@tools/generate_highlights.py` ? **the batch highlight RUNNER** (the clean,
orchestrated way to do "make reels for these videos into this folder").
`::generate_highlights_batch(video_paths, out_dir, *, segmenter, local_work_dir, db_path,
force)` + `::atomic_publish` (copy?`os.replace`; never deletes siblings) + `::resolve_videos`.
**Guarantees:** generates LOCALLY then publishes atomically into the dest (Drive-sync-safe ?
never generate inside a Google-Drive folder); **NEVER bulk-deletes** the dest; per-video
`clear_video_data` (idempotent ? no dup-rally accumulation; pairs with the compiler overlap-
merge); resumable (skip published reels unless `--force`); per-video `` - run.log` +
`_batch_run.log` published additively. CLI: `python -m tools.generate_highlights --videos-dir
<dir> --out <dir> --segmenter gemini [--force]` or repeated `--video`. ? WSL detectors
(wasb/tracknet) localize the source first (Drive invisible to /mnt). **Use THIS for batch reels
? never hand-roll a loop that writes into Drive or rm's the folder** (that lost finished reels +
run.logs once).
189 - `@tools/cleanup_caches.py` ? **top-level** `tools/` ops tool (NOT backend runtime).

```

```

Dry-run-first cache GC. Pure `::classify`(name,is_dir,location)/`::plan_deletions` (unit-tested
= the destructive decision path) + I/O `::scan_wsl` (du+find; ? arg-globbing NOT a `for` loop ?
the login shell empties loop vars) / `::scan_windows` / `::_delete`. CLI: `python -m
tools.cleanup_caches` (report) . `--apply` / `--keep-video <id|stem>` / `--older-than N` /
`--include-wasbcache` / `--summary` (hook nudge). Purges `*_frames`/staged/proxy/handoff/tmp;
keeps CSV/DB/reels. Companion to the runner self-clean (2.2). See `[[cache-hygiene-policy]]`
memory.
190     - `@backend/utils/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure =
"unknown"), `::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at
nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires
`reencode=True`.
191     - `@backend/utils/geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s;
? raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention
inflates IoU), `::get_center/get_distance/get_velocity`.
192
193     ### 2.5 Validators ?
194     - `@backend/pipeline/validators/base.py::VideoValidator` ABC, `::ValidationResult`
(pydantic: validator_name,passed,message,details), `::ValidationRegistry` (ordered LIST ?
registration order = exec order; ? no dedup), `::registry` (singleton). $ `validate(video_path,
config, context) -> ValidationResult`. `::run_all_with_context -> (results, context)` (wraps
each validate in try/except -> failed result not abort); `::run_all` discards context. ? **add a
validator**: subclass + `registry.register(MyValidator())`.
195     - `@backend/pipeline/validators/__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global
registry as side effect).
196     - `@backend/pipeline/validators/format.py::FormatValidator` (`format_check`) ?
**PRODUCER** of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is
substring `mp4` (so MOV-family passes); external `ffprobe`.
197     - `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if
absent). `min_fps` 29.9; min res 1280x720.
198     - `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator`
(`pts_continuity_check`) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded
(comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES
(too short).
199     - `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator`
(`scene_cut_check`) ? OpenCV. ? `cut_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps
at 100 SAMPLED frames (~first 50s); `max_scene_cuts_per_minute` (0.5) is the config fail
threshold.
200     - `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var <
`min_blur_threshold` (default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ?
magic absolute, not resolution-normalized.
201     - `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ?
HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`;
3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport
-> empty cfg, falls back to defaults, does NOT fail.
202
203     ### 2.6 Sports / Providers / Annotations ?
204     - `@backend/sports/base.py::SportProfile` ABC
(`name`,`get_prompt`,`get_relevance_config`);
`@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
`@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton; ?
`register` instantiates profile to read `name`). ? **add a sport**: subclass `SportProfile`,
`sport_registry.register(MyProfile)`. $ prompt emits JSON array `{start_time(float DECIMAL SEC),
end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt
actively defends against MM:SS; `get()` returns a fresh instance each call.
205     - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`);
`@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX_UPLOAD_MB=500`;
upload->poll->generate JSON temp 0->delete); `@backend/providers/__init__.py::provider_registry`
(+`::ProviderRegistry`, auto-registers `gemini`). ? **add a provider**: subclass,
`provider_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api_key,
model_name)`. $ `analyze_video(path, prompt, chunk_duration=0.0, label='') -> (segments:list,
metrics:dict{upload_time,process_time,gen_time})`. ? blocking 10s poll; returns `([], metrics)`
on ANY failure (empty != error); oversize clip refused LOUDLY (`logger.error` +

```

```

`metrics['oversize_skipped']=True` ? callers aggregate into the report's `oversize_clip_skipped`
rule); orphaned remote file possible if process dies mid-run.
206     - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`),
`::Interval` (= metrics.Interval), `::QUEUED/RUNNING/DONE/FAILED`/`::STATUSES`,
`::AnnotationProviderRegistry` (decorator-friendly `register` returns the class; ? keys off
`provider_cls().name`, falls back to `__name__` if no-arg ctor throws; `list_available()` ->
LIST not dict), `::annotation_registry`. ? **add a tier**: subclass (define `name`),
`annotation_registry.register(MyCls)` at module bottom + import in `__init__.py`. § `fetch` ->
List[Interval]` sorted; contract says RAISE (not `[ ]`) on unavailable.
207     - `@backend/annotations/file_provider.py::FileProvider` (`file`; job_id IS the
resolved CSV path; `_resolve` -> `</video_ref>.csv`; ->
`backend.eval.annotations.load_annotations`; ? `guideline`/`sport` ignored).
208     - `@backend/annotations/automated_provider.py::AutomatedProvider` (`auto`; GS-4 stub;
injectable `::Labeler`, in-process `_jobs` store, `::warn_if_same_lineage` ORACLE RULE ? only
warns, does not block; default `_not_configured` RAISES NotImplementedError). ? get via
`annotation_registry.get('auto', labeler=fn, lineage='...')`; registered as `auto` only
because no-arg ctor succeeds.
209     - `@backend/annotations/__init__.py` ? ? `noqa F401` imports of
file_provider/automated_provider are load-bearing (removing de-registers providers).
210
211     ### 2.7 Infrastructure (DB) ?
212     - `@backend/infrastructure/database.py::Database` ? SQLite, 6 tables
(`videos/play_segments/events/candidate_segments/jobs/user_models`), `PRAGMA user_version`
migration ladder in `::init_db` (currently ==5*: `version<1` creates `candidate_segments`;
`version<2` creates `jobs` + `idx_jobs_status` ? PR-1/#16; `version<3` ALTERS `jobs` to add
`policy`/`next_poll_at` + `idx_jobs_due` (self-scheduling EXECUTION_POLICY); `version<4` ALTERS
`videos`+`candidate_segments` to add a NULLABLE `user_id` (NULL=local install) + partial
`idx_videos_user`/`idx_candidates_user`; `version<5` creates `user_models` +
`idx_user_models_user_version`/`idx_user_models_one_active` (per-user model store,
PERSONALIZATION_PLAN ? persisted/loaded but NOT yet read on the served path)).
`::get_connection` (? re-issues `PRAGMA foreign_keys=ON` per connection ? else CASCADE/SET NULL
silently skipped). Methods: `add_video`, `add_play_segment`, `add_event`, `add_candidate_segment`,
`link_candidate_to_segment`, `query_candidate_segments(detector=,only_unlabeled=)`, `query_play_seg
ments`, `get_video`, `clear_video_data`, `segment_watermarks`, `prune_segments`; job store: `create_
job`, `mark_job_running`, `set_job_progress`, `set_job_video_id`, `mark_job_done(result)`, `mark_job_
failed(error)`, `get_job` (deserializes `result`/`player_pool` JSON), `fail_stale_jobs` (startup
sweep -> count). ? candidate_segments is the detector-agnostic fine-tuning extension point
(common fields columns, detector-specifics in `stats`/`detection_params` JSON; index
`idx_candidates_video_detector`). ? `add_video` is an **UPSERT** (`INSERT ... ON CONFLICT(id) DO
UPDATE`) that mutates the row in place ? it does NOT cascade-delete dependent segments on re-
ingest (that was the old `INSERT OR REPLACE` footgun); clearing segments is now an explicit
destroy-AFTER-confirm via `segment_watermarks`/`prune_segments` (or `clear_video_data`). Events
column is `type` not `event_type`; `query_play_segments` filters use truthiness so
`duration_min=0`/empty-string = "no filter"; write helpers swallow exceptions.
213
214     ### 2.8 Reporting / observability ? (per-video, soft-dep `obsreport`)
215     - `@backend/reporting/__init__.py` ? `::emit_indexer_report(*, db, video_id, video_path,
config, val_results, val_context=None, segmenter_requested=, segmenter_actual=,
was_reingest=False, segmentation_ran=True)` ? single entrypoint, keyword-only. Normalizes typed
config (`config.model_dump(...) if 'hasattr model_dump`); short-circuits to None if
unavailable/disabled; pulls play_segments/candidates from `db` (read-only
`query_play_segments(vid,{})`/`query_candidate_segments(vid)`); delegates to
`harvest.record_run`; writes via `rec.write(output_dir=config["report"]["output_dir"])`. ? wraps
everything in bare `except Exception: return None` ? **NEVER raises** (designed for `main.py`'s
`finally:`); writes `

```

```

{unknown,probed,fallback`}), `::validation_stage`, `::segmentation_stage` ($ `details.{segment
er_requested,segmenter_actual,config_default,matches_config_default,segmenter_explicitly_request
ed,was_reingest,detector,metadata_source}`; ? marks `motion` output "synthetic/heuristic"),
`::ai_stage` ($ `details.{ai_based,provider,model,api_key_present,oversize_clips_skipped}`;
metric `confirmed_segments`), `::highlights` (coverage_pct). `record_run(run_signals=)` carries
`run_signals.pop(video_id)` counters into `ai_stage`. ? `record_run` fps fallback:
`fps_source='unknown'` + ran -> promotes to "fallback", forces fps=30.0 (mirrors runners).
`::SPEC_PATH`/`::load_spec` resolve `spec.yaml` next to module. (Soft import of obsreport so
pure helpers like video_meta are importable for tests/coverage even without the dep.)
217 - `@backend/reporting/spec.yaml` ? $ declarative report layout + footgun engine consumed
by obsreport. `::customer_verdict` (pass/pass_with_warnings/fail messages). `::sections`
(ordered `{id,title,kind,audience?{both,customer,technical}}`). ? `::rules` ? each
`{code,severity,when[{path,op,value}],message,faq_ref,customer_message?}`; `when` clauses AND'd;
`path` dotted-navigates the recorder JSON. ? `path` strings are a HARD coupling to harvest's
stage/metric/detail names ? renaming a detail in harvest.py silently breaks the matching rule.
Rules (9): `silent_provider_noop` (error: no api key + 0 segs), `oversize_clip_skipped` (error:
provider refused chunks over MAX_UPLOAD_MB ? 4K stream-copy footgun),
`ai_empty_vs_no_rallies` (warn), `fps_fallback_used` (warn), `audio_disabled_or_missing` (warn: no
audio stream -> loses the shuttle-thwack recall cue; reads `video_meta.audio_present`),
`synthetic_motion_metadata` (warn), `segmenter_divergence` (info),
`reingest_replaced_prior` (info), `validation_failed` (error: halts indexing).
218 - `@backend/reporting/run_signals.py` ? ? in-process, thread-safe channel pipeline-
internals -> report (`::incr(video_id,key,by)`, `::pop(video_id)`). No obsreport dep (always
importable). Segmenter worker threads record facts the harvest can't see (e.g.
`oversize_clips_skipped`); `emit_indexer_report` pops UNCONDITIONALLY (even reporting-off, so
nothing accumulates) and passes to `record_run`. ? process-local best-effort: signals are
dropped (by design) if no report is emitted for that video_id.
219
220 ---
221
222 ## 3. DATA CONTRACTS (canonical shapes ? one place)
223
224 $ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by
`parse_trajectory_csv`):
225 `Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel
coords.
226 -> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).
227
228 $ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
`_extract_action_windows_from_chunk`):
229 `{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.
230
231 $ **candidate_segments DB row** (`db.add_candidate_segment` /
`query_candidate_segments`):
232 `{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL,
duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count},
detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}`
(stats/detection_params deserialized on read).
233 `detection_params` keys by detector:
wasb={detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}; tracknet
adds `queue_length`; yolo={velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
234
235 $ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
236 `{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.
237
238 $ **jobs row** (`db.create_job` / `get_job` ? #16, PR-1): `{id:TEXT(uuid4.hex),
video_path, video_id?:TEXT(set once hashed), segmenter?, player_pool?:JSON list, max_frames?,
status:'queued'|'running'|'done'|'failed' (EXECUTION state ? pipeline verdict is
result["status"]), progress?:TEXT(stage label), error?:TEXT, result?:JSON(the former sync

```

```

/api/process response incl. reports), created_at, started_at?, finished_at?}`.
239
240     $ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time,
end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start;
`_candidate_id` stamped internally).
241
242     $ **GT / golden CSVs** (TWO conventions ? do not conflate):
243     - generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal
or MM:SS/HH:MM:SS; RAISES on bad row.
244     - golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
245
246     $ **det_raw.jsonl** (one window/line): `{ "w":<window_index>,
"f":[[frame_id,[x,y,score,scale],...], ...]}`. Lines MUST be contiguous (line N has `w=N`);
first violation/JSONDecodeError -> truncate-heal.
247     $ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
248     $ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv},
...]}`; ? DB key = file MD5, manifest `video_id` is a label.
249
250     $ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.
251
252     $ **WSL detector candidate** (in-WSL, model<->tracker): `{ 'xy':np.array([x,y]),
'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST
rebuild `xy` as np.array (tracker does `np.linalg.norm`).
253
254     $ **HitEvent** (audio, `pipeline/audio/hit_detector`): `{t:float(sec), strength:float}`
? onset-envelope peak.
255
256     $ **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages
`validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail};
flags `{code,faq_ref}`; `video_meta.fps_source` ? {probed,fallback}. ? stage/metric/detail names
are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync:
`ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmenta
tion.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinges
t}`, `validation.status`, `video_meta.fps_source`.
257
258     ---
259
260     ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ?
claims below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
261     - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra
`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is
single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::build_cfg.)
262     - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH),
ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read
from cfg["model"] at runtime in wasb_infer.run.)
263     - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats)` -> (results,
hms_vis). `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
264     - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes
every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()`
resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls
`build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y`
? consistent.)
265     - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh,
transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)

```

```

266     - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-
data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial,
avoided.
267
268     ---
269
270     ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified
here)
271     - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-
encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest
tracks the exact file used.
272     - `python -m src.cli.curate export` (collector) -> `//.csv`
(indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` +
`@scripts/run_phase3_eval.py`.
273     - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
274
275     ---
276
277     ## 6. GOTCHAS (cross-cutting footguns)
278     - **MD5 keying:** `video_id` = MD5 of WHOLE file via the single
`@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all
run_*.py. Manifest `video_id` is a label, NOT the DB key.
279     - **Config access:** `backend/main.py` and all four `run_*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config/.indexing` is `None` (the ASGI-import edge case).
280     - **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.
281     - **`AppConfig.get` must return dict sections, not models:** the legacy
`.get("indexing",{}).get(...)` chains everywhere depend on it; the `extra="allow"` model also
silently keeps typo'd config keys (no rejection).

```

### 13. user (2026-06-16T02:25:12.201Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27

```

```

28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46 boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55 Any]]:
56             """Return (runner, detector-specific detection_params extras)."""
57             raise NotImplementedError
58
59         @staticmethod
60         def _probe_fps_width(video_path: str) -> tuple:
61             """Return (fps, frame_width) for a video, with sensible fallbacks."""
62             cap = cv2.VideoCapture(video_path)
63             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65             cap.release()
66             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68         @staticmethod
69         def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70             """Provider-reported exchange count, else a duration-based estimate.
71
72             One canonical implementation ? the twins had drifted (wasb relied on
73             ``int(None)`` raising into the fallback; same result, by accident).
74             """
75             shot_count = segment.get("shuttle_exchange_count")
76             if shot_count is None:
77                 return max(3, int(duration / 0.8))
78             try:
79                 return int(shot_count)
80             except (ValueError, TypeError):
81                 return max(3, int(duration / 0.8))
82
83         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84         -----
85         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86 frame_width: float, video_path: str,
87 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89 BASE
90 returns exactly the 4 motion keys, so the trajectory twins persist byte-
91 identical
92 rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue

```

```

88         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
89         return {
90             "peak_velocity": cw["peak_velocity"],
91             "mean_velocity": cw["mean_velocity"],
92             "active_frame_count": cw["active_frame_count"],
93             "track_id_count": cw["track_id_count"],
94         }
95
96     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                             window_stats: List[Dict[str, Any]],
98                             idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                       player_pool: Optional[List[str]] = None,
108                       max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
123         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
124         # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
125         # measure it against the Gemini path, or to avoid the cloud entirely). With
126         # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
127         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
128         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
129         if skip_ai:
130             model_name = "local-noai"
131             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
132                         "be emitted as rallies; no %s handoff, no API key needed.",
133                         label, provider_name)
134         else:
135             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
136             api_key_env_var = f"{provider_name.upper()}_API_KEY"
137             api_key = os.environ.get(api_key_env_var)
138             if not api_key:
139                 logger.error(
140                     f"{api_key_env_var} environment variable is not set! "
141                     f"To run the {provider_name} handoff, set your API key. "
142                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
143                 )

```

```

144         return []
145     try:
146         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
147     except KeyError as e:
148         logger.error(str(e))
149         return []
150
151     video_duration = get_video_duration(video_path)
152     if video_duration <= 0:
153         logger.error("Invalid video duration.")
154         return []
155     fps, frame_width = self._probe_fps_width(video_path)
156
157     # --- Runner config + windowing thresholds ---
158     runner, runner_params = self._build_runner(idx_cfg)
159
160     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
161     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
162     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
163     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
164     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
165     log_candidates = idx_cfg.get("log_yolo_candidates", True)
166     store_candidates = idx_cfg.get("store_yolo_candidates", True)
167
168     detection_params = {
169         "detector": self.name,
170         "velocity_thresh": velocity_thresh,
171         "merge_gap": merge_gap,
172         "min_action_window_duration": min_window_duration,
173         **runner_params,
174     }
175
176     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
177     ok, msg = runner.healthcheck()
178     if not ok:
179         logger.error("%s WSL environment not ready: %s", label, msg)
180         return []
181     logger.info("%s WSL env OK: %s", label, msg)
182
183     # --- Phase 2: trajectory -> candidate rally windows ---
184     # Trajectory detectors process the whole video in one pass (they carry
185     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
186     t_start = time.time()
187     out_dir = os.path.join("output", self.family)
188     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
189     if not csv_path:
190         logger.error("%s produced no trajectory; aborting.", label)
191         return []
192
193     points = runner.parse_trajectory_csv(csv_path)
194     logger.info("Parsed %d trajectory points (%d visible).",
195               len(points), sum(1 for p in points if p.visible))
196
197     all_candidate_windows = runner.trajectory_to_action_windows(
198         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
199         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
200         min_window_duration=min_window_duration, chunk_index=0,
201     )
202     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
203               label, time.time() - t_start, len(all_candidate_windows))
204
205     if log_candidates:
206         for cw in all_candidate_windows:
207             logger.info(f"[{label}] rally")

```



```

260
261     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
262         w_start = max(0, window["start"] - handoff_padding)
263         w_end = min(video_duration, window["end"] + handoff_padding)
264         w_dur = w_end - w_start
265         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
266         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
267             return []
268         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
269         segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
270                                             label=f"Handoff {wi+1}")
271         valid = []
272         for seg in segments:
273             try:
274                 seg["start_time"] = float(seg["start_time"]) + w_start
275                 seg["end_time"] = float(seg["end_time"]) + w_start
276                 seg["_candidate_id"] = candidate_ids[wi]
277                 valid.append(seg)
278             except (ValueError, TypeError, KeyError) as e:
279                 logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
280             try:
281                 os.remove(w_path)
282             except OSError:
283                 pass
284         return valid
285
286         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
287             futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
288                         for wi in handoff_indices]
289             for future in futures:
290                 ai_segments.extend(future.result())
291         try:
292             os.rmdir(handoff_dir)
293         except OSError:
294             pass
295         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
296
297         # --- Phase 4: DB population + candidate linking ---
298         segments_data = []
299         for segment in ai_segments:
300             try:
301                 start = float(segment["start_time"])
302                 end = float(segment["end_time"])
303                 dur = end - start
304                 metadata = {
305                     "shot_count": self._shot_count_from(segment, dur),
306                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
307                     "detector": f"{self.family}-hybrid-{model_name}",
308                 }
309                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
310                 if segment_id:
311                     candidate_id = segment.get("_candidate_id")
312                     if candidate_id is not None:
313                         self.db.link_candidate_to_segment(candidate_id, segment_id)
314                 segments_data.append({
315                     "id": segment_id, "start_time": start, "end_time": end,
316                     "duration": dur, "metadata": metadata,

```

```

317         })
318     except Exception as e:
319         logger.warning("Skipping invalid segment struct: %s", e)
320
321     return segments_data
322

```

#### 14. user (2026-06-16T02:25:12.597Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5     2. copies wasb_infer.py into the WASB repo's src/ dir,
6     3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7     4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21    TrackNetRunner.
22    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
24    wsl_tilde_quote
25
26    logger = logging.getLogger(__name__)
27
28    # Windows path to the inference wrapper we stage into WSL.
29    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
30
31    @dataclass
32    class WasbConfig:
33        """Resolved settings for a WASB WSL run (built from config.json ->
34        indexing.wasb)."""
35        distro: str = "Ubuntu"
36        repo_dir: str = "~/models/WASB-SBDT"
37        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
38        conda_env: str = "wasb"
39        weights_path: str = "~/models/WASB-
34    SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
35        sport: str = "badminton"
36        wsl_stage_dir: str = "~/clips"
37        timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
38        keep_frames: bool = False # retain staged video + frame cache after success (debug
39        only)
40        min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
41        off)
42
43        # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44        # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45        # extraction that dominates a long clip. Output is bit-identical to the PNG path
46        # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
47        it.
48        stream_video: bool = False
49
50    @classmethod

```

```

50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                  f"python -c \"import torch; print('torch', torch.__version__, "
76                  f"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."
83             out = (res.stdout or "").strip()
84             if res.returncode != 0:
85                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86             return True, out
87
88         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89             src_mnt = to_wsl_mnt_path(video_win_path)
90             reason = self.wsl_source_unreadable_reason(src_mnt)
91             if reason:
92                 logger.error("Cannot stage video into WSL: %s", reason)
93                 return None
94             dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96             try:
97                 res = self._wsl_bash(cmd)
98             except subprocess.TimeoutExpired:
99                 logger.error("Staging video into WSL timed out.")
100                 return None
101             if res.returncode != 0 or "OK" not in (res.stdout or ""):
102                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103                 return None
104             return dest
105
106         def _stage_infer_script(self) -> bool:
107             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108             src_mnt = to_wsl_mnt_path(_INFER_WIN)
109             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110             res = self._wsl_bash(cmd)

```

```

111         return res.returncode == 0 and "OK" in (res.stdout or "")
112
113     def run_predict(self, video_win_path: str, output_win_dir: str,
114                    video_id: Optional[str] = None) -> Optional[str]:
115         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
116
117         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
118         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
119         can never reuse each other's staged frames, resumable cache, or CSV.
120         """
121         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
122         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
123         # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
124         # the CSV landed inside WSL while Windows polled the repo-relative path.
125         output_win_dir = os.path.abspath(output_win_dir)
126         os.makedirs(output_win_dir, exist_ok=True)
127         # Normalize for cross-platform basename (tests use Windows paths on Linux).
128         video_win_path = str(video_win_path).replace("\\", "/")
129         base = os.path.basename(video_win_path)
130         stem, ext = os.path.splitext(base)
131         # Unique, content-derived key: same filename + different bytes -> different key.
132         key = f"{stem}__{video_id[:12]}" if video_id else stem
133         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136         if not self._stage_infer_script():
137             logger.error("Could not stage wasb_infer.py into WSL.")
138             return None
139         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140         if staged_video is None:
141             return None
142         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145         # WSL stage filesystem is already low so the user can clean up before it fills.
146         # (Streaming never materializes the PNGs, so the guard only matters off the fast
147         path.)
148         if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
149             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
150             if free is not None and free < self.cfg.min_free_gb:
151                 logger.warning(
152                     "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
153                     extraction "
154                     "may fill the disk ? consider running tools/cleanup_caches.py.",
155                     free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
156
157         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
158         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
159         clip.
160         if self.cfg.stream_video:
161             frame_args = "--stream-video "
162         else:
163             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
164         cmd = (
165             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
166             f"python wasb_infer.py "
167             f"--video {wsl_tilde_quote(staged_video)} "
168             f"{frame_args}"
169             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
170             f"--sport {shlex.quote(self.cfg.sport)} "
171             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}")
172         )
173         logger.info("Running WASB inference on %s ...", base)
174         try:
175             res = self._wsl_bash(cmd)

```

```

173         except subprocess.TimeoutExpired:
174             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175             return None
176         if res.returncode != 0:
177             logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178             return None
179         if not os.path.exists(expected_csv_win):
180             logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181             return None
182         logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184         # Success ? the trajectory CSV is the durable output; the staged video copy and
185         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187         # so the frame cache + detector cache remain for a resumable re-run.
188         # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189         if not self.cfg.keep_frames:
190             stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191             logger.info("Cache hygiene: removing staged video%s for %s "
192                         "(set indexing.wasb.keep_frames=true to retain).",
193                         "" if self.cfg.stream_video else " + frame cache", key)
194             self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

## 15. assistant (2026-06-16T02:25:17.397Z)

I now have the core data contracts. Let me read the remaining pieces: fusion\_features, rally\_gate, fusion\_hybrid, the database candidate/play\_segment persistence methods, hashing, and the compiler. Let me confirm exact field shapes by reading the source.

## 16. user (2026-06-16T02:25:18.392Z)

```

1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of scale/translation-invariant
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12     - per_frame is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13     index (so it aligns directly with the shuttle trajectory's frame ? no clip re-
timing).
14     - A shuttle point has frame, visible, x, y (pixels) ? the WASB
trajectory shape.
15     - court_polygon is normalised 0-1 [(x, y), ...] or None (None ? no mask,
every
16     detection counts ? the player-count-only mode).

```

```

17 - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
18 0.0).
19
20 Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
21 floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
22 """
23
24 from __future__ import annotations
25
26 from typing import Dict, List, Optional, Sequence, Tuple
27
28 BBox = Tuple[float, float, float, float]
29 Net = Tuple[str, float]
30
31 def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
32     """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
33     (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
34     if polygon is None or len(polygon) < 3:
35         return False
36     x, y = point
37     inside = False
38     n = len(polygon)
39     j = n - 1
40     for i in range(n):
41         xi, yi = polygon[i]
42         xj, yj = polygon[j]
43         # Does the horizontal ray at y cross edge (i, j)?
44         if (yi > y) != (yj > y):
45             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
46             if x < x_cross:
47                 inside = not inside
48         j = i
49     return inside
50
51 def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52     """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53     the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54     if frame_width <= 0 or frame_height <= 0:
55         return (0.0, 0.0)
56     x1, _y1, x2, y2 = box
57     return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60 def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61     if frame_width <= 0 or frame_height <= 0:
62         return (0.0, 0.0)
63     x1, y1, x2, y2 = box
64     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67 def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69     """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70     True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71     point meaningless, so a masked check returns False rather than testing a bogus
72     (0,0)."""
73     if court_polygon is None:
74         return True
75     if frame_width <= 0 or frame_height <= 0:
76         return False
77     return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)

```

```

77
78
79     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                          frame_height: float,
81                          court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82     """Per-frame count of in-court persons (one entry per sampled frame)."""
83     return [
84         sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
85         for _frame_idx, dets in sorted(per_frame.items())
86     ]
87
88
89     def _median(vals: Sequence[float]) -> float:
90         if not vals:
91             return 0.0
92         s = sorted(vals)
93         n = len(s)
94         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97     def players_in_court(counts: Sequence[int]) -> float:
98         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99         return _median(list(counts))
100
101
102     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
104         if not counts:
105             return 0.0
106         return sum(1 for c in counts if c >= min_players) / len(counts)
107
108
109     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                           frame_width: float, frame_height: float) -> float:
111         """Mean per-track centre displacement between consecutive sampled frames, with x
112         normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113         not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114         if frame_width <= 0 or len(per_frame) < 2:
115             return 0.0
116         frames = sorted(per_frame.keys())
117         steps: List[float] = []
118         for a, b in zip(frames, frames[1:]):
119             prev = {tid: box for tid, box in per_frame[a]}
120             for tid, box in per_frame[b]:
121                 if tid in prev:
122                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125         if not steps:
126             return 0.0
127         return sum(steps) / len(steps)
128
129
130     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                          frame_height: float, net: Optional[Net],
132                          court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137         0.0.
138         """

```

```

138     if net is None or not per_frame:
139         return 0.0
140     axis, pos = net
141     both = 0
142     for _frame_idx, dets in per_frame.items():
143         near, far = False, False
144         for _tid, box in dets:
145             if not person_in_court(box, frame_width, frame_height, court_polygon):
146                 continue
147             fx, fy = _foot_norm(box, frame_width, frame_height)
148             coord = fx if axis == "x" else fy
149             if coord < pos:
150                 near = True
151             else:
152                 far = True
153         if near and far:
154             both += 1
155     return both / len(per_frame)
156
157
158     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                        pad_frac: float) -> bool:
160         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161         width/height (the 'adjacent / in-hand' tolerance)?"""
162         x1, y1, x2, y2 = box
163         px = pad_frac * frame_width
164         py = pad_frac * frame_height
165         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                                   frame_width: float, frame_height: float,
170                                   pad_frac: float = 0.03) -> float:
171         """Fraction of (visible shuttle frames that were also person-sampled) in which the
172         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
173         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
174         """
175         if not per_frame or frame_width <= 0 or frame_height <= 0:
176             return 0.0
177         aligned = 0
178         colocated = 0
179         for p in shuttle_points or []:
180             if not getattr(p, "visible", False):
181                 continue
182             dets = per_frame.get(int(p.frame))
183             if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
184                 continue
185             aligned += 1
186             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187                   for _tid, box in dets):
188                 colocated += 1
189         return colocated / aligned if aligned else 0.0
190
191
192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199

```

```

200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                                     frame_width: float, frame_height: float,
202                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203                                     net: Optional[Net] = None,
204                                     min_players: int = 2) -> Dict[str, float]:
205     """Aggregate the per-window person features from per-frame detections + the in-
window
206     shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208     return {
209         "players_in_court": players_in_court(counts),
210         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212         "in_court_ratio": in_court_ratio(counts, min_players),
213         "shuttle_player_colocation": shuttle_player_colocation(
214             per_frame, shuttle_points, frame_width, frame_height),
215     }
216

```

## 17. user (2026-06-16T02:25:19.321Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Optional, Sequence, Tuple
24
25    Interval = Tuple[float, float]
26
27
28    @dataclass
29    class GatedWindow:
30        start: float
31        end: float
32        crossings: int
33        visible_points: int
34        kept: bool
35
36
37    def _spread(vals: Sequence[float]) -> float:
38        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39        if len(vals) < 2:
40            return 0.0
41        s = sorted(vals)

```

```

42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06,
93                 estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94     """Annotate each (start,end) window with its shuttle net-crossing count and a
95     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96     axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98     `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99     `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100     per window ? same result, avoids the per-window re-estimation."""
101     axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102     deadband = deadband_frac * span
103     out: List[GatedWindow] = []
104     for (s, e) in windows:

```

```

105         wp = _points_in(points, fps, s, e)
106         c = count_net_crossings(wp, axis, net, deadband)
107         vis = sum(1 for p in wp if p.visible)
108         out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                               kept=c >= min_crossings))
110     return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

## 18. user (2026-06-16T02:25:19.769Z)

```

1     """Multi-signal fusion segmenter (MULTI-SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13    torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19    **Persisted candidates remain the full superset** regardless, so the offline
experiment
20    harness can re-score any variant.
21
22    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26    experiment harness (EXPERIMENT_HARNESS.md), never silently.
27    """
28    from typing import Any, Dict, List, Optional, Tuple
29
30    from backend.pipeline.segmenters.base import segmenter_registry
31    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32    from backend.pipeline.detectors.base import DetectorRunner
33    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35    from backend.pipeline.detectors import rally_gate
36    from backend.pipeline.detectors import fusion_features as ff
37
38

```

```

39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48             self._person: Optional[Any] = None
49             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50             # over the whole trajectory once per candidate window (it depends only on
`points`).
51             self._axis_cache_key: Optional[int] = None
52             self._axis_cache: Optional[tuple] = None
53
54         @property
55         def name(self) -> str:
56             return "fusion_hybrid"
57
58         @property
59         def description(self) -> str:
60             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                     "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                     "AI provider sets semantic boundaries.")
63
64         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66             if substrate == "tracknet":
67                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                 return TrackNetRunner(cfg), {
69                     "weights_path": cfg.weights_path,
70                     "queue_length": cfg.queue_length,
71                     "fusion_substrate": "tracknet",
72                 }
73             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                     frame_width: float, video_path: str,
78                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82             # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83             gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                                  min_crossings=0,
estimate=self._axis_estimate(points))
85             stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87             fcfg = idx_cfg.get("fusion", {})
88             if fcfg.get("cue_flags", {}).get("person", False):
89                 stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))

```

```

90         return stats
91
92     # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93     # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94     # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95     # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96     # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97     # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98     # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100    def _axis_estimate(self, points: List[Any]) -> tuple:
101        """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
102        computed once per video, not once per candidate window (it depends only on
points)."""
103        key = id(points)
104        if self._axis_cache_key != key or self._axis_cache is None:
105            self._axis_cache_key = key
106            self._axis_cache = rally_gate.estimate_play_axis(points)
107        est = self._axis_cache
108        assert est is not None
109        return est
110
111    def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
frame_width: float, video_path: str,
112                        fcfg: Dict[str, Any]) -> Dict[str, float]:
113        """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
114        frame range and compute the scale/translation-invariant person features. Lazily
builds
115        ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
116        # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
117        from backend.pipeline.detectors.person_detector import PersonDetector
118        if self._person is None:
119            self._person = PersonDetector(self.config)
120
121        frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
122        # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
123        # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
124        start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
125        det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
126        fw = det["frame_width"] or frame_width
127        fh = det["frame_height"]
128        # If we can't trust the frame dims, every normalised feature would silently
collapse
129        # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
130        if fw <= 0 or fh <= 0:
131            return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
132
133        poly = fcfg.get("court_polygon")
134        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
135        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
136        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
137        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
court_polygon=polygon, net=net)

```

```

140
141     def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                             window_stats: List[Dict[str, Any]],
143                             idx_cfg: Dict[str, Any]) -> List[int]:
144         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145         (0) keep every window ? identical to the substrate. Persisted candidates are
146         untouched (full superset)."""
147         fcfg = idx_cfg.get("fusion", {})
148         min_crossings = int(fcfg.get("min_crossings", 0))
149         min_players = int(fcfg.get("min_players", 0))
150         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
151
152         keep: List[int] = []
153         for i, st in enumerate(window_stats):
154             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
155                 continue
156             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
157                 continue
158             keep.append(i)
159         return keep
160
161
162     segmenter_registry.register(FusionSegmenter)
163

```

## 19. user (2026-06-16T02:25:20.272Z)

```

1     """Canonical file-identity hashing for the indexer.
2
3     `video_id` is the MD5 of the *whole* video file. Historically each entryptoint
4     (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
5     `scripts/run_process_and_stitch.py`)
6     defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
7     The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
8     any future change to one copy would silently diverge the DB key.
9
10    This module is the single source of truth. Import `compute_video_id` everywhere a
11    video needs to be keyed.
12    """
13
14    from __future__ import annotations
15
16    import hashlib
17
18    # 64 KB matches the value used by the live pipeline (backend/main.py) before this
19    # was centralized. The chunk size does not affect the resulting digest ? it only
20    # bounds memory use while streaming large files.
21    _CHUNK_SIZE = 65536
22
23    def compute_video_id(filepath: str) -> str:
24        """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26        Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27        into memory at once.
28        """
29        hasher = hashlib.md5()
30        with open(filepath, "rb") as f:
31            for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
32                hasher.update(chunk)
33        return hasher.hexdigest()
34

```

## 20. assistant (2026-06-16T02:25:23.589Z)

I now have a complete picture of the data contracts. Let me read the database persistence methods (candidate\_segments, play\_segments) for exact column shapes, and the compiler, to nail down the final-segment contract and the stitch transformation.

## 21. user (2026-06-16T02:25:25.038Z)

```
1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):
30             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31             # adds concurrent writers (a swallowed 'database is locked' silently drops
32             segments).
33             conn = sqlite3.connect(self.db_path, timeout=30.0)
34             conn.row_factory = sqlite3.Row
35             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36             conn.execute("PRAGMA foreign_keys = ON")
37             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38             conn.execute("PRAGMA busy_timeout = 30000")
39             try:
40                 conn.execute("PRAGMA journal_mode = WAL")
41             except sqlite3.OperationalError:
42                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
43             return conn
44
45         def _init_db(self):
46             """Initializes tables for videos, play_segments, and events."""
47             with self._connect() as conn:
48                 cursor = conn.cursor()
49
50                 # Videos Table
51                 cursor.execute("""
52                     CREATE TABLE IF NOT EXISTS videos (
53                         id TEXT PRIMARY KEY,
54                         filepath TEXT NOT NULL,
55                         processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
56                         status TEXT NOT NULL,
57                         validation_results TEXT
```

```

57         )
58     """)
59
60     # Play Segments Table (Extensible metadata JSON)

```

## 22. user (2026-06-16T02:25:26.759Z)

```

291:     def add_play_segment(self, video_id: str, start_time: float, end_time: float, server:
Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]] = None)
-> Optional[int]:
318:     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
341:     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
355:     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
389:     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,
Any]]:
439:     def clear_video_data(self, video_id: str):

```

## 23. user (2026-06-16T02:25:30.921Z)

```

60     # Play Segments Table (Extensible metadata JSON)
61     cursor.execute("""
62         CREATE TABLE IF NOT EXISTS play_segments (
63             id INTEGER PRIMARY KEY AUTOINCREMENT,
64             video_id TEXT NOT NULL,
65             start_time REAL NOT NULL,
66             end_time REAL NOT NULL,
67             duration REAL NOT NULL,
68             server TEXT,
69             receiver TEXT,
70             metadata TEXT,
71             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
72         )
73     """)
74
75     # Events Table
76     cursor.execute("""
77         CREATE TABLE IF NOT EXISTS events (
78             id INTEGER PRIMARY KEY AUTOINCREMENT,
79             segment_id INTEGER NOT NULL,
80             timestamp REAL NOT NULL,
81             type TEXT NOT NULL,
82             player TEXT,
83             details TEXT,
84             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
85         )
86     """)
87
88     # Versioned migrations live here. PRAGMA user_version is the schema
89     # contract ? bump it for every change and add an ordered `if version < N`
90     # block below. Tests assert the expected version, so accidental drift fails
91     CI.
92     version = cursor.execute("PRAGMA user_version").fetchone()[0]
93
94     if version < 1:
95         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
96         # Common fields are columns; detector-specific metrics go in `stats`
97         JSON,
98         # so a new segmenter reuses this table by setting its own
99         `detector`/`stats`.
100         cursor.execute("""
            CREATE TABLE IF NOT EXISTS candidate_segments (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                video_id TEXT NOT NULL,

```

```

101         detector TEXT NOT NULL,
102         chunk_index INTEGER,
103         start_time REAL NOT NULL,
104         end_time REAL NOT NULL,
105         duration REAL NOT NULL,
106         confidence REAL,
107         stats TEXT,
108         detection_params TEXT,
109         confirmed_segment_id INTEGER,
110         human_label TEXT,
111         notes TEXT,
112         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
113         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
114         FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
115     )
116     """)
117     cursor.execute("""
118         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
119         ON candidate_segments(video_id, detector)
120     """)
121     cursor.execute("PRAGMA user_version = 1")
122
123     if version < 2:
124         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
125         # /api/process request. `status` is the EXECUTION state of the job
126         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
127         # that failed validation) lives inside `result` JSON as
result["status"].
128         # `result` is the full sync-era response dict (incl. report paths), so
the
129         # observability report is the job's artifact, per the plan.
130         cursor.execute("""
131             CREATE TABLE IF NOT EXISTS jobs (
132                 id TEXT PRIMARY KEY,
133                 video_path TEXT NOT NULL,
134                 video_id TEXT,
135                 segmenter TEXT,
136                 player_pool TEXT,
137                 max_frames INTEGER,
138                 status TEXT NOT NULL DEFAULT 'queued',
139                 progress TEXT,
140                 error TEXT,
141                 result TEXT,
142                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
143                 started_at TIMESTAMP,
144                 finished_at TIMESTAMP
145             )
146             """)
147         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
148         cursor.execute("PRAGMA user_version = 2")
149
150     if version < 3:
151         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
152         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
153         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
154         # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
155         # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
156         cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")

```

```

157         cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
158         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
159         cursor.execute("PRAGMA user_version = 3")
160
161     if version < 4:
162         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
163         # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
164         # path can scope rows to an owner. **NULL = local install = byte-
identical to
165         # today** ? every existing row stays NULL and every existing query
166         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
167         # Additive ONLY: no served-behavior change rides on this slice.
168         cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
169         cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
170         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
171         # of index churn while making the eventual per-user lookups cheap.
172         cursor.execute(
173             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
174             "WHERE user_id IS NOT NULL")
175         cursor.execute(
176             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
177             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
178         cursor.execute("PRAGMA user_version = 4")
179
180     if version < 5:
181         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
(user_id,
182         # version): the resolved per-user `fusion_config` overlay + an INLINE
183         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
184         # and provenance that earned it. `is_active` pins the one row the
serving bridge
185         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
This slice
186         # only persists/loads it ? NOTHING reads it on the served path yet.
187         #
188         #   user_id           ? owner of the model (NULL allowed = the local
install)
189         #   version           ? monotonic per-user model version
190         #   baseline_version   ? the shared baseline this was fit against
(upgrade switch)
191         #   feature_schema_hash ? fingerprint of the feature set; MINOR-vs-
MAJOR re-fit gate
192         #   fusion_config      ? JSON per-user FusionConfig overlay (cue_flags
/ thresholds)
193         #   classifier_json    ? JSON LogisticRegression.to_dict() (None =
config-only model)
194         #   promotion_evidence  ? JSON per_user_gate PromotionDecision (why it
was promoted)
195         #   n_videos_at_fit     ? held-out-user corpus size when fit (min_n
trust floor input)
196         #   is_active          ? the one resolved row for this user (partial-
unique below)
197         cursor.execute("""
198             CREATE TABLE IF NOT EXISTS user_models (
199                 id INTEGER PRIMARY KEY AUTOINCREMENT,
200                 user_id TEXT,
201                 version INTEGER NOT NULL DEFAULT 1,
202                 baseline_version TEXT,

```

```

203         feature_schema_hash TEXT,
204         fusion_config TEXT,
205         classifier_json TEXT,
206         promotion_evidence TEXT,
207         n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
208         is_active INTEGER NOT NULL DEFAULT 0,
209         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
210     )
211     """
212     # One model version per user (NULLs compare distinct in SQLite, so the
local
213     # install can still carry multiple versioned rows; the active-row guard
below
214     # is what enforces a single served model).
215     cursor.execute("""
216         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
217         ON user_models(user_id, version)
218     """)
219     # At most ONE active model per user: a partial-unique index over the
active rows.

```

#### 24. user (2026-06-16T02:25:31.474Z)

```

291     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
292         try:
293             with self._connect() as conn:
294                 cursor = conn.cursor()
295                 cursor.execute(
296                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
297                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
298                 )
299                 conn.commit()
300                 return cursor.lastrowid
301         except Exception as e:
302             logger.error(f"Failed to add play segment: {e}")
303             return None
304
305     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
306         try:
307             with self._connect() as conn:
308                 conn.cursor().execute(
309                     "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
310                     (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
311                 )
312                 conn.commit()
313                 return True
314         except Exception as e:
315             logger.error(f"Failed to add event: {e}")
316             return False
317
318     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
319                             chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
320                             stats: Optional[Dict[str, Any]] = None,
321                             detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
322         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are

```

```

323         detector-specific dicts stored as JSON. Returns the new candidate id, or
324         None."""
325         try:
326             with self._connect() as conn:
327                 cursor = conn.cursor()
328                 cursor.execute(
329                     """INSERT INTO candidate_segments
330                        (video_id, detector, chunk_index, start_time, end_time, duration,
331                         confidence, stats, detection_params)
332                        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
333                        (video_id, detector, chunk_index, start_time, end_time, end_time -
334                         start_time,
335                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
336                         {})))
337                 conn.commit()
338                 return cursor.lastrowid
339             except Exception as e:
340                 logger.error(f"Failed to add candidate segment: {e}")
341                 return None
342
343     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
344         """Record that a candidate detection was confirmed as a given play_segment."""
345         try:
346             with self._connect() as conn:
347                 conn.cursor().execute(
348                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
349                     ?",
350                     (segment_id, candidate_id)
351                 )
352                 conn.commit()
353                 return True
354             except Exception as e:
355                 logger.error(f"Failed to link candidate {candidate_id} to segment
356                 {segment_id}: {e}")
357                 return False
358
359     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
360                                only_unlabeled: bool = False) -> List[Dict[str, Any]]:
361         """Fetch candidate detections for the annotation / fine-tuning workflow.
362         `stats` and `detection_params` are deserialized from JSON."""
363         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
364         params: List[Any] = [video_id]
365
366         if detector:
367             query += " AND detector = ?"
368             params.append(detector)
369         if only_unlabeled:
370             query += " AND human_label IS NULL"
371
372         query += " ORDER BY start_time"
373
374         candidates = []
375         with self._connect() as conn:
376             rows = conn.cursor().execute(query, params).fetchall()
377             for row in rows:
378                 d = dict(row)
379                 d["stats"] = json.loads(d["stats"] or "{}")
380                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
381                 candidates.append(d)
382         return candidates
383
384     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
385         with self._connect() as conn:
386             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",

```

```

(video_id,)).fetchone()
383         if row:
384             d = dict(row)
385             d["validation_results"] = json.loads(d["validation_results"] or "[]")
386             return d
387         return None
388
389     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
390         """
391         Dynamically queries play segments based on filters:
392         - duration_min: float
393         - server: string
394         - receiver: string
395         - event_type: string (e.g. smash, foul)
396         """
397         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
398         params = [video_id]
399
400         if filters.get("duration_min"):
401             query += " AND r.duration >= ?"
402             params.append(filters["duration_min"])
403
404         if filters.get("server"):
405             query += " AND r.server = ?"
406             params.append(filters["server"])
407
408         if filters.get("receiver"):
409             query += " AND r.receiver = ?"
410             params.append(filters["receiver"])
411
412         if filters.get("event_type"):
413             # Check if there is an event of a specific type inside this segment
414             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
415             params.append(filters["event_type"])
416
417         segments_list = []
418         with self._connect() as conn:
419             cursor = conn.cursor()
420             rows = cursor.execute(query, params).fetchall()
421
422             for row in rows:
423                 r_dict = dict(row)
424                 r_id = r_dict["id"]
425                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
426
427                 # Fetch associated events
428                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
429                 r_dict["events"] = []
430                 for er in event_rows:
431                     e_dict = dict(er)
432                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
433                     r_dict["events"].append(e_dict)
434
435                 segments_list.append(r_dict)
436
437         return segments_list
438
439     def clear_video_data(self, video_id: str):
440         with self._connect() as conn:
441             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
442             conn.commit()
443

```

```

444 # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
445 # Writers follow the repo convention (swallow + logger.error, return bool); the
446 # worker logs loudly on a failed transition so a job can't silently wedge.
447
448 def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
449                 player_pool: Optional[List[str]] = None,
450                 max_frames: Optional[int] = None,

```

## 25. assistant (2026-06-16T02:25:36.959Z)

I now have everything needed. Let me read the compiler's segment-selection and stitch logic to confirm the final transformation and where the video file is touched, plus how main.py selects segment time-pairs for compilation.

## 26. user (2026-06-16T02:25:39.340Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     logger = logging.getLogger(__name__)
9
10    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13    # backend/pipeline/stitcher/, so parents[2] == backend/.
14    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16    class VideoCompiler:
17        def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                    watermark: Optional[Any] = None):
19            self.output_dir = output_dir
20            self.end_padding = end_padding
21            self.begin_padding = begin_padding
22            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23            # normalized to a dict (or None) so this module stays config-package-agnostic.
24            self.watermark = self._normalize_watermark(watermark)
25            os.makedirs(output_dir, exist_ok=True)
26
27    @staticmethod
28    def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29        if watermark is None:
30            return None
31        if hasattr(watermark, "model_dump"):
32            return watermark.model_dump()
33        if isinstance(watermark, dict):
34            return dict(watermark)
35        return None
36
37    @staticmethod
38    def _ff_quote(path: str) -> str:
39        """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40        filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41        slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43        The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44        option parser treats ':' as the option separator, so a Windows drive-letter

```

```

45         path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46         the encode fails with "Invalid argument". This silently disabled the
47         on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48         return path.replace("\\", "/").replace("'", r'\').replace(":", r"\:")
49
50     @staticmethod
51     def _parse_time(val: Union[int, float, str]) -> float:
52         """Normalizes a timestamp value to float seconds.
53         Handles: float/int (pass-through), plain numeric strings ("12.4"),
54         and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55         This mirrors the parse_time logic used in gemini.py to ensure
56         the stitcher can always consume whatever format the indexer produced."""
57         if isinstance(val, (int, float)):
58             return float(val)
59         if isinstance(val, str):
60             val = val.strip()
61             if ":" in val:
62                 # Handle MM:SS, HH:MM:SS, etc.
63                 parts = val.split(":")
64                 parts.reverse()
65                 total = 0.0
66                 for i, p in enumerate(parts):
67                     try:
68                         total += float(p) * (60 ** i)
69                     except ValueError:
70                         pass

```

## 27. user (2026-06-16T02:25:40.149Z)

```

81:     def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float, float]]:
190:     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float, float]],
output_filename: str) -> str:
201:         segments = self._merge_overlapping(segments)

```

## 28. user (2026-06-16T02:25:46.069Z)

```

81     def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
82         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
83
84         A highlight reel must never replay the same footage. The same rally can land in
85         the segment list more than once ? e.g. two detector runs accumulated for one
86         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
87         or
88         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
89         rally
90         twice. Merging any range that overlaps or abuts the previous one collapses true
91         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
92         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
93         """
94         parsed = []
95         for raw_s, raw_e in segments:
96             s = VideoCompiler._parse_time(raw_s)
97             e = VideoCompiler._parse_time(raw_e)
98             if e > s:
99                 parsed.append((s, e))
100         parsed.sort()
101         merged: List[Tuple[float, float]] = []
102         for s, e in parsed:
103             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
104                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
105             else:
106                 merged.append((s, e))
107         return merged

```

```

107 def _get_video_duration(self, video_path: str) -> float:
108     """Probes the video file to get its total duration in seconds.
109     Returns 0.0 if it cannot be determined."""
110     try:
111         result = subprocess.run(
112             [
113                 "ffprobe", "-v", "error",
114                 "-show_entries", "format=duration",
115                 "-of", "default=noprint_wrappers=1:nokey=1",
116                 video_path
117             ],
118             capture_output=True, text=True, check=True
119         )
120         return float(result.stdout.strip())
121     except Exception as e:
122         logger.warning(f"Could not determine video duration via ffprobe: {e}")
123         return 0.0
124
125 def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
126     """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
127     and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
128     watermarking is disabled or has nothing to draw. The concat stage stays -c copy
129     because the mark is already baked into every clip identically."""
130     wm = self.watermark
131     if not wm or not wm.get("enabled"):
132         return None
133
134     text = (wm.get("text") or "").strip()
135     logo_path = (wm.get("logo_path") or "").strip()
136     if logo_path and not os.path.exists(logo_path):
137         logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
138 overlay.")
139         logo_path = ""
140     if not text and not logo_path:
141         return None
142
143     margin = int(wm.get("margin", 24))
144     opacity = float(wm.get("opacity", 0.85))
145     position = str(wm.get("position", "bottom-right"))
146     # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
147     w/h (logo).
148     dt_xy = {
149         "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
150         "bottom-left": f"x={margin}:y=h-th-{margin}",
151         "top-right": f"x=w-tw-{margin}:y={margin}",
152         "top-left": f"x={margin}:y={margin}",
153     }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
154     ov_xy = {
155         "bottom-right": f"W-w-{margin}:H-h-{margin}",
156         "bottom-left": f"{margin}:H-h-{margin}",
157         "top-right": f"W-w-{margin}:{margin}",
158         "top-left": f"{margin}:{margin}",
159     }.get(position, f"W-w-{margin}:H-h-{margin}")
160
161     drawtext = ""
162     if text:
163         ratio = float(wm.get("font_size_ratio") or 0.035)
164         divisor = max(1, round(1.0 / ratio))
165         # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
166         strings.
167         tf_path = os.path.join(tmp_dir, "watermark.txt")
168         with open(tf_path, "w", encoding="utf-8") as fh:
169             fh.write(text)
170         # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
171         # without fontconfig) ? fontconfig family (last-resort, environment-

```

```

dependent).
169         font_path = (wm.get("font_path") or "").strip()
170         if not font_path and _BUNDLED_FONT.exists():
171             font_path = str(_BUNDLED_FONT)
172         if font_path:
173             font_opt = f"fontfile='{self._ff_quote(font_path)}'"
174         else:
175             font_opt = f"font='{wm.get('font', 'Sans')}'"
176         box = ""
177         if wm.get("box", True):
178             box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
179         drawtext = (
180             f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
181             f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
182         )
183
184         if logo_path and drawtext:
185             return
186         f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
187         if logo_path:
188             return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
189         return drawtext
190
191     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
192 float]], output_filename: str) -> str:
193         """
194         Extracts selected segments from a raw video and stitches them together
195         into a seamless highlights file, applying end_padding to prevent abrupt endings.
196         """
197         if not segments:
198             raise ValueError("No highlight segments provided to compile.")
199
200         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
201         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
202         original_count = len(segments)
203         segments = self._merge_overlapping(segments)
204         if len(segments) < original_count:
205             logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
206 -> %d.",
207                             original_count - len(segments), original_count, len(segments))
208         if not segments:
209             raise ValueError("No valid (start < end) highlight segments after merge.")
210
211         if not os.path.exists(raw_video_path):
212             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
213 {raw_video_path}")

```

## 29. user (2026-06-16T02:25:47.763Z)

```

31:from backend.pipeline.stitcher.compiler import VideoCompiler
562:def query_play_segments(req: QueryRequest):
569:    segments = db_instance.query_play_segments(req.video_id, filters)
573:def compile_highlights(req: CompileRequest):
579:    all_segments = db_instance.query_play_segments(req.video_id, {})
616:    time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
623:    out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
758:    res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)

```

## 30. assistant (2026-06-16T02:25:52.016Z)

I have the complete picture now, including the final compile path. Let me read main.py around the compile/process flow to confirm exactly where segments are read from DB and turned into time-pairs (the pure DB?reel transformation), confirming the video-free seam between

segmentation and stitching.

### 31. user (2026-06-16T02:25:53.037Z)

```
573     def compile_highlights(req: CompileRequest):
574         video = db_instance.get_video(req.video_id)
575         if not video:
576             raise HTTPException(status_code=404, detail="Indexed video record not found.")
577
578         # Retrieve matching play segments from db
579         all_segments = db_instance.query_play_segments(req.video_id, {})
580         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
581
582         if not matched_segments:
583             raise HTTPException(status_code=400, detail="No matching play segments found to
584 stitch.")
585
586         # Reject path traversal / absolute output names (os.path.join would otherwise let an
587         # absolute or '..'-laden output_filename write anywhere on disk).
588         try:
589             out_name = safe_output_filename(req.output_filename)
590         except ValueError as e:
591             raise HTTPException(status_code=400, detail=str(e)) from e
592
593         # The configured shot-count floor (stitching.min_shot_count) applies on the API
594         # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
595         shot_count
596         # metadata are exempt: the floor filters known counts only, so explicitly
597         # requested manual/legacy segments can't silently vanish under the default floor.
598         stitching = getattr(global_config, "stitching", None) or StitchingConfig()
599         kept = []
600         for s in matched_segments:
601             meta = s.get("metadata") or {}
602             sc = meta.get("shot_count") if isinstance(meta, dict) else None
603             try:
604                 if sc is not None and int(sc) < stitching.min_shot_count:
605                     continue
606             except (TypeError, ValueError):
607                 pass # unparseable count -> treat as unknown, keep
608             kept.append(s)
609         if len(kept) < len(matched_segments):
610             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
611 dropped "
612 f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
613 segments below the floor.")
614         if not kept:
615             raise HTTPException(status_code=400,
616 detail=f"All {len(matched_segments)} matching segments fall
617 below "
618 f"stitching.min_shot_count={stitching.min_shot_count}.")
619
620         # Extract start/end time pairs
621         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
622
623         # Run compiler with the configured stitching paddings + optional branding watermark
624         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
625         or "output"
626         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
627 end_padding=stitching.end_padding,
628 watermark=stitching.watermark)
629         try:
630             out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
631             return {"status": "success", "filepath": out_path}
632         except Exception as e:
```

```
626             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
627
```

**32. assistant (2026-06-16T02:25:56.572Z)**

I now have the complete end-to-end picture with all concrete field names confirmed against source. Let me compile the structured answer.

**33. user (2026-06-16T02:27:33.139Z)**

Structured output provided successfully

**34. assistant (2026-06-16T02:27:36.599Z)**

I've mapped the pipeline's data contracts and identified the video-free seam. The structured output has been provided successfully with the complete analysis covering the four requested areas: the end-to-end stage sequence with exact artifact shapes, where video/frames are required vs pure transformation, the best freeze seams for video-free regression, and sources of non-determinism downstream.